



Машинное обучение: от парадигм к практике

Структурированная карта предметной области для начинающего ML-инженера: от базовых парадигм и классических алгоритмов — через архитектуры глубокого обучения — к практическому плану первых шагов и системе метрик оценки качества моделей.

Содержание лекции

01

Парадигмы машинного обучения

Supervised, Unsupervised и Reinforcement learning — три базовых типа обратной связи от среды.

03

Архитектуры глубокого обучения

MLP, CNN, RNN/LSTM, Transformer — структура, задачи, ключевые нюансы каждой архитектуры.

02

Базовые модели ML

Линейная и логистическая регрессия, деревья решений, k-NN — плюсы, минусы, области применения.

04

Практика и метрики

Три шага для старта и система метрик оценки качества классификации и регрессии.

Три парадигмы машинного обучения

Всё машинное обучение делится на три базовые парадигмы по типу доступной обратной связи от среды. Выбор парадигмы определяет, какие данные нужны, как строится обучение и какие задачи можно решать.



Supervised Learning — обучение с учителем

Принцип работы

Модель обучается на парах «вход — правильный ответ» (размеченные данные). На каждом шаге обучения модель получает входные данные, делает предсказание и сравнивает его с известным правильным ответом, корректируя свои параметры для уменьшения ошибки.

Пример задачи

Предсказание цены квартиры по её характеристикам (площадь, район, этаж) на основе исторических данных о проданных квартирах с известной ценой. Здесь каждый обучающий пример — это квартира с набором признаков и известной ценой продажи.

Типичные задачи

- Классификация (спам / не спам)
- Регрессия (предсказание числового значения)
- Детекция объектов на изображениях

Unsupervised Learning — обучение без учителя

Принцип работы

Модель находит структуру в данных без разметки. Нет «правильных ответов» — алгоритм самостоятельно обнаруживает скрытые паттерны, группировки или компактные представления данных.

Пример задачи

Сегментация клиентов интернет-магазина на группы по покупательскому поведению без заранее заданных категорий (кластеризация). Алгоритм сам определяет, сколько групп выделить и по каким признакам их разграничить.

Типичные задачи

- Кластеризация (k-means, DBSCAN)
- Снижение размерности (PCA, t-SNE)
- Обнаружение аномалий

Reinforcement Learning — обучение с подкреплением

Принцип работы

Агент учится через взаимодействие со средой, получая награду или штраф за свои действия. Цель — выработать стратегию (policy), максимизирующую суммарную накопленную награду на длинном горизонте.

Пример задачи

Обучение робота-манипулятора захватывать объекты произвольной формы через серию проб и ошибок с наградой за успешный захват. Робот не получает инструкций — он сам исследует пространство действий и постепенно улучшает стратегию.

Типичные задачи

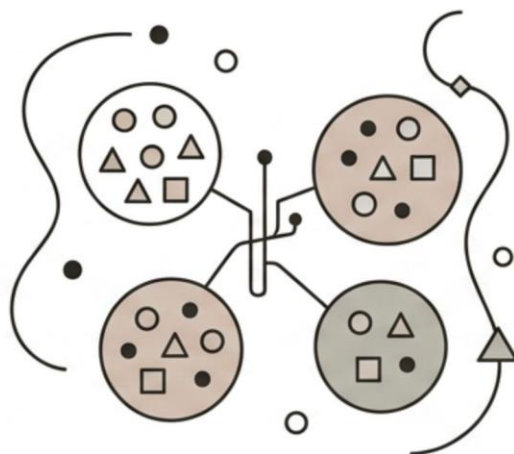
- Управление роботами
- Игровые агенты (AlphaGo, OpenAI Five)
- Оптимизация рекомендательных систем

Сравнение трёх парадигм



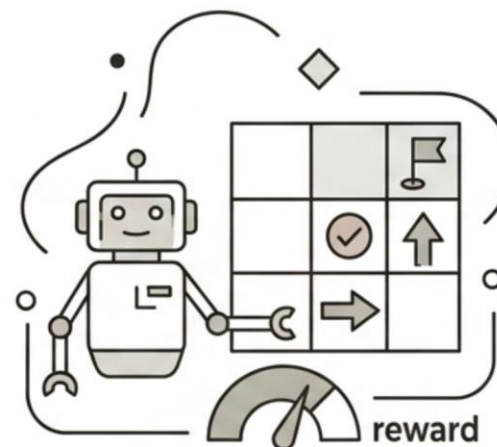
**Размеченные
данные**
**Предсказание
результата**

Примеры:
Классификация,
Регрессия



**Данные без
меток**
**Поиск
структуры**

Примеры:
Кластеризация,
Понижение размерности



Агент и среда
**Сигнал
награды**

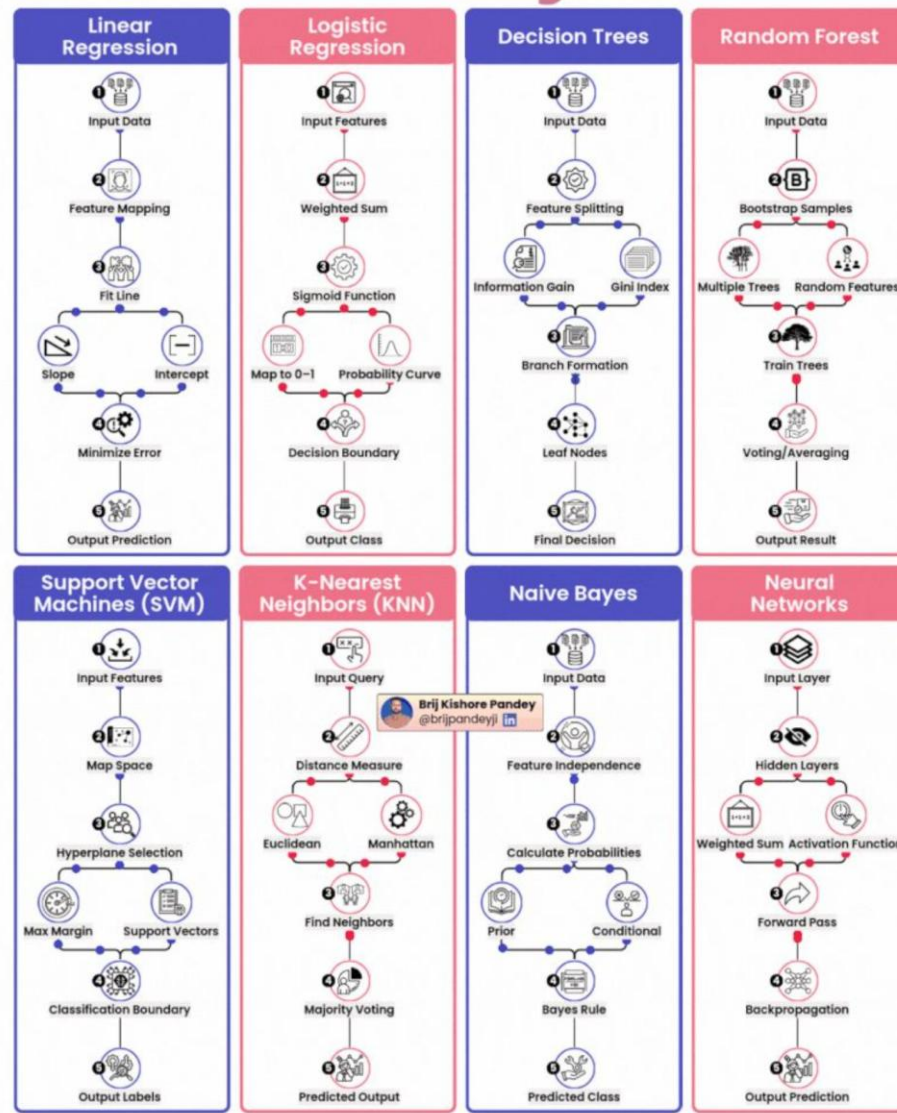
Примеры:
Робототехника,
Игры

РАЗДЕЛ 2

Базовые модели машинного обучения

Классические алгоритмы, которые должен знать каждый ML-инженер. Они остаются актуальными как baseline-решения и как инструменты для задач, где интерпретируемость важнее сложности.

8 ML Algorithms You Should Always Know



Линейная регрессия

Тип задачи

Регрессия — предсказание непрерывного числового значения.

Модель строит линейную зависимость между входными признаками и целевой переменной: $\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.

Параметры w подбираются минимизацией среднеквадратичной ошибки.

Плюсы

- Очень быстрое обучение и инференс
- Высокая интерпретируемость: каждый коэффициент показывает вклад признака
- Хорошо работает при линейных зависимостях
- Не требует больших вычислительных ресурсов

Минусы

- Плохо работает при нелинейных зависимостях в данных
- Чувствительна к выбросам (из-за квадратичной функции потерь)
- Требуется масштабирования признаков для корректной регуляризации

Логистическая регрессия

Тип задачи

Классификация — обычно бинарная. Несмотря на название, это алгоритм классификации: линейная комбинация признаков пропускается через сигмоид-функцию, которая преобразует результат в вероятность принадлежности к классу: $P(y=1) = \sigma(w^T x) = 1/(1+e^{-w^T x})$.

Плюсы

- Выдаёт вероятности, а не просто метки классов
- Легко интерпретируется через коэффициенты
- Быстрое обучение, хорошо масштабируется
- Эффективна при линейно разделимых классах

Минусы

- Строит только линейную решающую границу
- Не справляется со сложными нелинейными паттернами без ручного создания признаков

Деревья решений

Тип задачи

Классификация и регрессия. Модель строит иерархическую структуру из правил «если-то»: на каждом узле дерева данные разбиваются по одному признаку, ветви ведут к дочерним узлам, листья содержат итоговые предсказания. Разбиение выбирается для максимального уменьшения неопределённости (Gini, энтропия).

Плюсы

- Не требуют масштабирования признаков
- Легко интерпретируются и визуализируются
- Работают с категориальными и числовыми признаками
- Улавливают нелинейные зависимости

Минусы

- Склонны к переобучению на шумных данных
- Нестабильны: небольшое изменение данных может сильно изменить дерево
- Одиночное дерево уступает ансамблям (Random Forest, Gradient Boosting)

k-NN (k ближайших соседей)

Тип задачи

Классификация и регрессия. Алгоритм не строит явную модель: при предсказании для нового объекта он находит k ближайших к нему объектов в обучающей выборке (по евклидовому или другому расстоянию) и возвращает большинство классов (для классификации) или среднее значение (для регрессии).

Плюсы

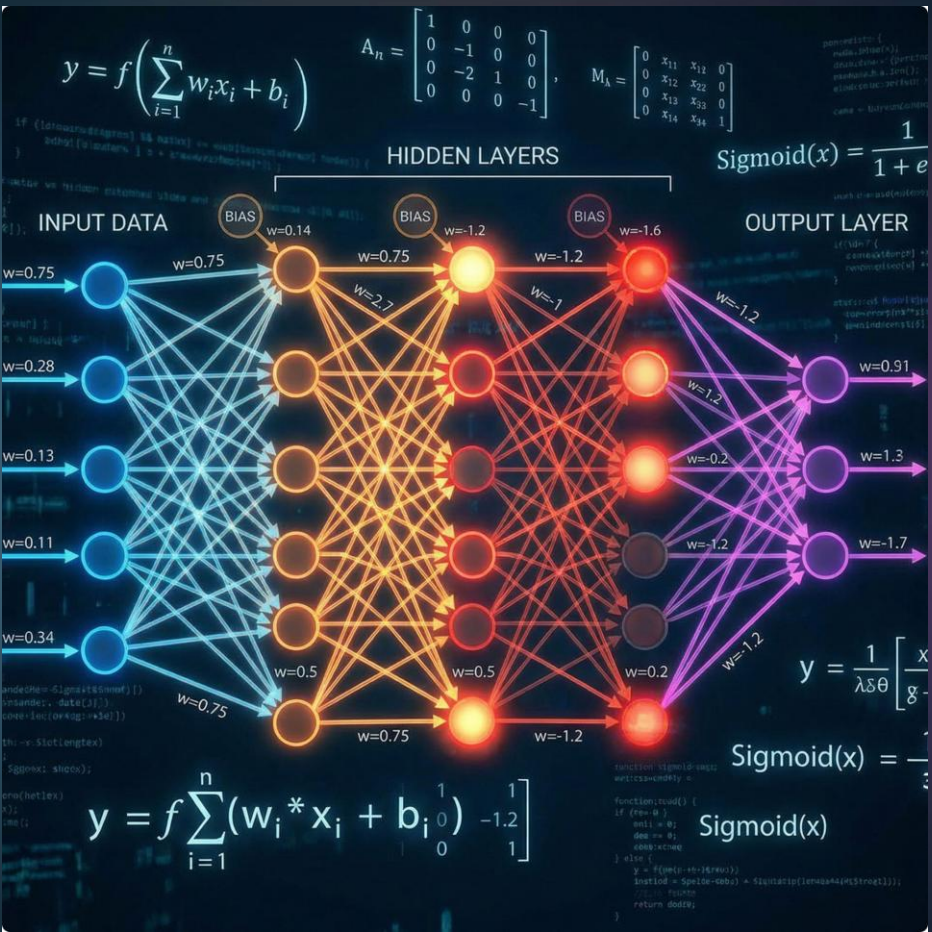
- Простая реализация без явного этапа обучения
- Легко адаптируется к новым данным (просто добавить в выборку)
- Естественно обрабатывает многоклассовые задачи

Минусы

- Медленный инференс: нужно вычислять расстояния до всех точек
- Чувствительность к масштабу признаков на больших датасетах
- Плохо работает в высокоразмерных пространствах (проклятие размерности)

Сводная таблица базовых моделей ML

Модель	Тип задачи	Плюсы	Минусы
Линейная регрессия	Регрессия	Быстрая, интерпретируемая	Только линейные зависимости
Логистическая регрессия	Классификация (бинарная)	Вероятности, интерпретируемость	Линейная граница решения
Деревья решений	Классификация и регрессия	Без масштабирования, интерпретируемы	Склонны к переобучению
k-NN	Классификация и регрессия	Простота, без явного обучения	Медленный инференс, чувствителен к масштабу



РАЗДЕЛ 3

Архитектуры глубокого обучения

Четыре ключевые архитектуры, которые лежат в основе современного глубокого обучения. Каждая решает специфический класс задач и обладает уникальными структурными свойствами.

Обзор архитектур: от MLP к Transformer

1

MLP

Полносвязные слои. Табличные данные.

2

CNN

Свёрточные слои. Изображения.

3

RNN / LSTM

Рекуррентные слои. Последовательности.

4

Transformer

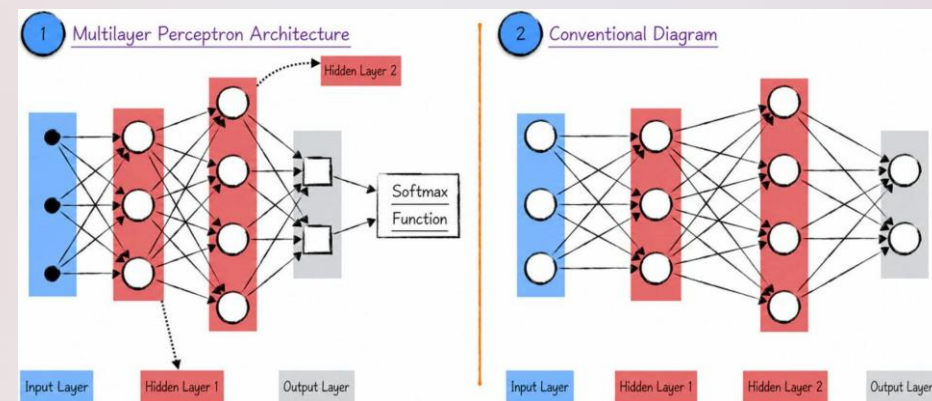
Self-attention. Текст и не только.

Каждая архитектура — это ответ на конкретный тип структуры данных: пространственную (CNN), временную (RNN/LSTM) или произвольные попарные зависимости (Transformer). MLP — универсальная основа без специализации.

АРХИТЕКТУРА 1

MLP — многослойный перцептрон

Фундаментальная архитектура глубокого обучения, лежащая в основе всех остальных.



MLP: структура и принцип работы

Структура

Последовательность полносвязных (dense) слоёв, где каждый нейрон связан со всеми нейронами предыдущего слоя. Между слоями стоит нелинейная функция активации (ReLU, sigmoid). Данные проходят строго последовательно: **вход** → **скрытые слои** → **выход**, без какой-либо специальной обработки пространственной или временной структуры.

Типичные задачи

- Табличные данные
- Простая классификация и регрессия на структурированных признаках
- Финальные слои в более сложных архитектурах (CNN, Transformer)

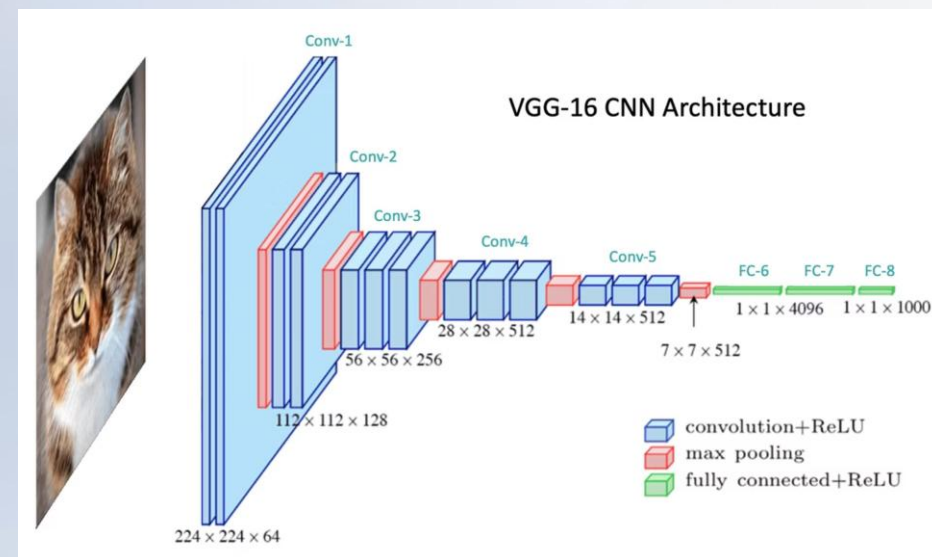
Ключевой нюанс

MLP не учитывает порядок или пространственное расположение признаков. Если входные данные переставить местами (перемешать столбцы таблицы одинаково для всех примеров), результат обучения не изменится. Это делает MLP неэффективным для изображений и последовательностей по сравнению со специализированными архитектурами.

АРХИТЕКТУРА 2

CNN — свёрточная нейронная сеть

Архитектура, специально разработанная для обработки данных с пространственной структурой — прежде всего изображений.



CNN: структура и принцип работы

Структура

Чередование **свёрточных слоёв** (применяют небольшие фильтры, скользящие по изображению и извлекающие локальные признаки) и **слоёв пулинга** (уменьшают размерность, сохраняя важную информацию). Завершается полносвязными слоями для итогового решения. Данные проходят как многомерный тензор (высота × ширина × каналы), сохраняя пространственную структуру на всех промежуточных этапах.

Типичные задачи

- Классификация изображений
- Детекция объектов
- Семантическая сегментация

Ключевой нюанс

За счёт **разделяемых весов фильтра** по всему изображению CNN обладает свойством трансляционной инвариантности — объект распознаётся независимо от того, в каком месте кадра он находится. Это кардинально снижает число обучаемых параметров по сравнению с MLP на том же изображении.

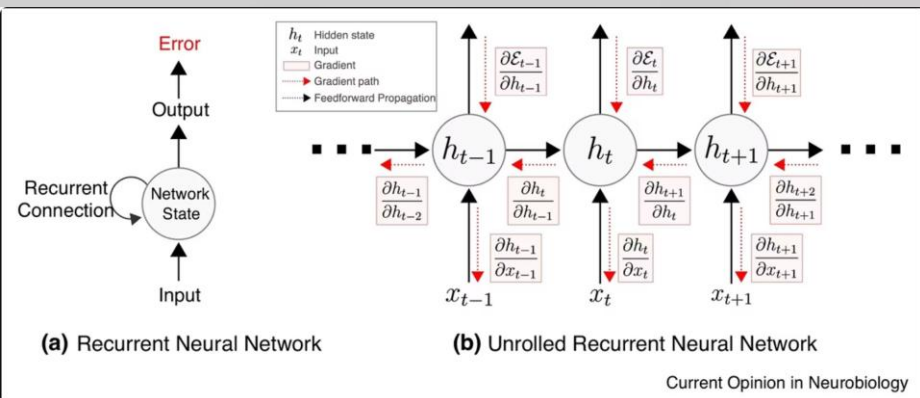
CNN vs MLP: почему свёртки эффективнее для изображений

MLP на изображении

Изображение $224 \times 224 \times 3 = 150\,528$ **входных нейронов**. Каждый нейрон первого скрытого слоя связан со всеми входами. При 1000 нейронах в слое — 150 миллионов параметров только в первом слое. Пространственная структура полностью теряется.

CNN на изображении

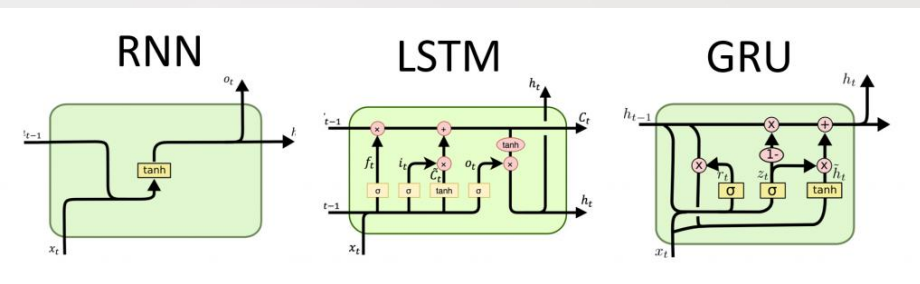
Фильтр $3 \times 3 \times 3 = 27$ **параметров**, применяется ко всему изображению. 64 таких фильтра = 1728 параметров в первом слое. Пространственная структура сохраняется. Трансляционная инвариантность — бесплатно.



АРХИТЕКТУРА 3

RNN / LSTM — рекуррентные сети

Архитектуры для обработки последовательных данных, где порядок элементов несёт смысловую нагрузку.



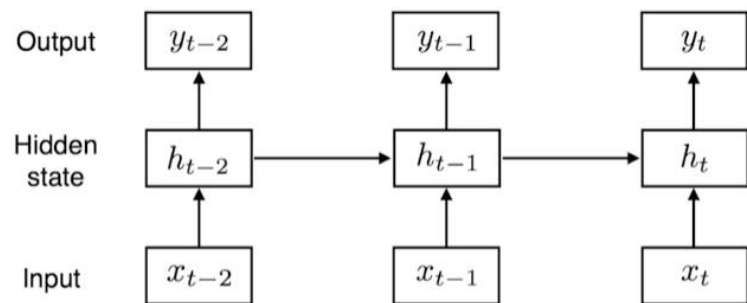
RNN: структура и проблема затухающего градиента

Структура RNN

Один и тот же блок сети применяется последовательно к каждому элементу входной последовательности. Скрытое состояние h_t передаётся от шага к шагу, аккумулируя информацию о предыдущих элементах:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

Типичные задачи: анализ текста, временные ряды, распознавание речи — везде, где важен порядок элементов.



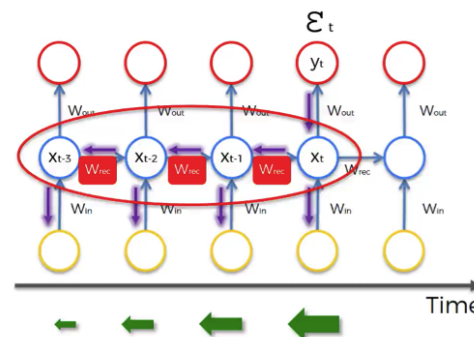
$$\begin{aligned} h_1 &= \phi(W^T h_0 + U^T x_1) \\ h_2 &= \phi(W^T \phi(W^T h_0 + U^T x_1) + U^T x_2) \\ h_3 &= \phi(W^T \phi(W^T \phi(W^T h_0 + U^T x_1) + U^T x_2) + U^T x_3) \\ h_4 &= \phi(W^T \phi(W^T \phi(W^T \phi(W^T h_0 + U^T x_1) + U^T x_2) + U^T x_3) + U^T x_4) \end{aligned}$$

Vanishing / exploding gradient

Проблема затухающего градиента

При обратном распространении ошибки через много шагов градиент умножается на одни и те же веса многократно и **экспоненциально уменьшается**. Из-за этого сеть «забывает» далёкий контекст — информация из начала длинной последовательности практически не влияет на обновление весов. Это фундаментальное ограничение базовой RNN, которое решает LSTM.

The Vanishing Gradient Problem



$$\frac{\partial \mathcal{E}}{\partial \theta} = \sum_{1 \leq t \leq T} \frac{\partial \mathcal{E}_t}{\partial \theta} \quad (3)$$

$$\frac{\partial \mathcal{E}_t}{\partial \theta} = \sum_{1 \leq k \leq t} \left(\frac{\partial \mathcal{E}_t}{\partial x_t} \frac{\partial x_t}{\partial x_k} \frac{\partial x_k}{\partial \theta} \right) \quad (4)$$

$$\frac{\partial x_t}{\partial x_k} = \prod_{t \geq k+1} \frac{\partial x_t}{\partial x_{t-1}} = \prod_{t \geq k+1} W_{rec}^T \text{diag}(\sigma'(x_{t-1})) \quad (5)$$

$W_{rec} \sim \text{small} \Rightarrow \text{Vanishing}$
 $W_{rec} \sim \text{large} \Rightarrow \text{Exploding}$

Formula Source: Razvan Pascanu et al. (2013)

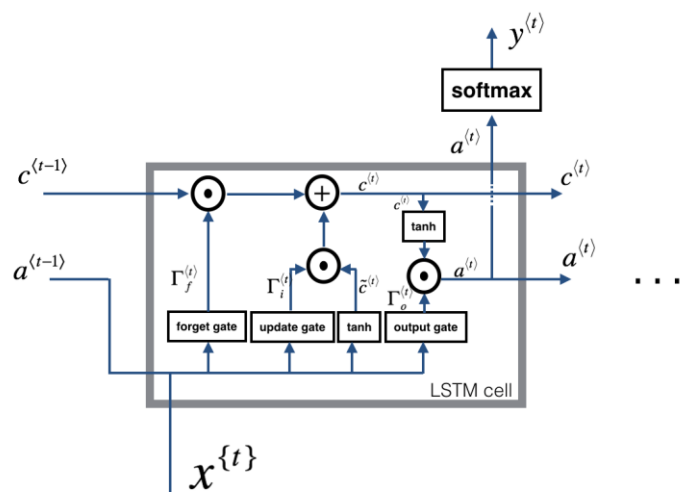
LSTM: решение проблемы затухающего градиента

Механизм гейтов

LSTM вводит отдельное «**клеточное состояние**» c_t с тремя гейтами (forget, input, output gate), которое обновляется преимущественно через сложение, а не полную перезапись:

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$$

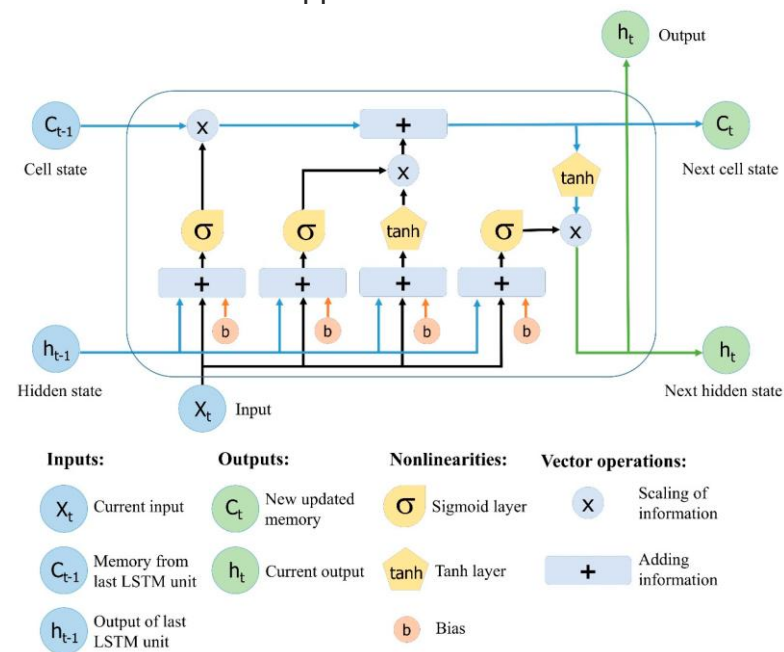
Где гейты f_t, i_t — обучаемые значения от 0 до 1.



$$\begin{aligned}\Gamma_f^{(t)} &= \sigma(W_f[a^{(t-1)}, x^{(t)}] + b_f) \\ \Gamma_u^{(t)} &= \sigma(W_u[a^{(t-1)}, x^{(t)}] + b_u) \\ \tilde{c}^{(t)} &= \tanh(W_c[a^{(t-1)}, x^{(t)}] + b_c) \\ c^{(t)} &= \Gamma_f^{(t)} \circ c^{(t-1)} + \Gamma_u^{(t)} \circ \tilde{c}^{(t)} \\ \Gamma_o^{(t)} &= \sigma(W_o[a^{(t-1)}, x^{(t)}] + b_o) \\ a^{(t)} &= \Gamma_o^{(t)} \circ \tanh(c^{(t)})\end{aligned}$$

Почему это работает

Аддитивное обновление клеточного состояния позволяет градиенту проходить через много шагов без катастрофического затухания — в отличие от полностью мультипликативного обновления в обычной RNN. Гейты обучаются решать, какую информацию **забыть** (forget gate), какую **добавить** (input gate) и что **передать на выход** (output gate). Это позволяет LSTM удерживать контекст на сотни шагов назад.



RNN vs LSTM: сравнение механизмов памяти

Базовая RNN

Единственное скрытое состояние h_t . Обновление полностью мультипликативное. Градиент затухает экспоненциально при длинных последовательностях. Практически не удерживает контекст дальше 10–20 шагов назад.

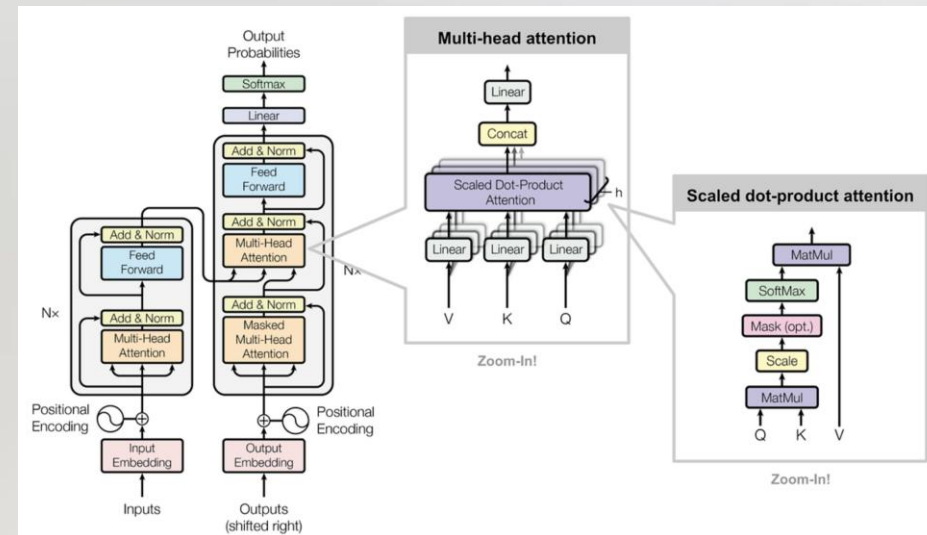
LSTM

Два состояния: скрытое h_t и клеточное c_t . Три обучаемых гейта управляют потоком информации. Аддитивное обновление клеточного состояния. Удерживает контекст на сотни шагов. Значительно сложнее по числу параметров.

АРХИТЕКТУРА 4

Transformer

Революционная архитектура, ставшая основой современных языковых моделей (GPT, BERT) и всё активнее применяемая за пределами NLP.



Transformer: механизм self-attention

Структура

Вместо последовательной обработки токенов, как в RNN, Transformer обрабатывает всю последовательность **параллельно** через механизм self-attention, который вычисляет, насколько каждый элемент последовательности «важен» для каждого другого:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Слои self-attention чередуются с полносвязными feed-forward блоками. Позиционные кодировки добавляются к входу, чтобы компенсировать отсутствие естественного порядка обработки.

Типичные задачи

- Машинный перевод
- Генерация текста (GPT)
- Языковые модели (BERT)
- Vision Transformer (ViT) — изображения

Ключевой нюанс

В отличие от RNN, где обработка строго последовательна и не параллелизуется по времени, Transformer обрабатывает всю последовательность за один проход. Это делает обучение значительно быстрее на GPU/TPU и позволяет напрямую «видеть» связи между далёкими элементами без риска затухания градиента.

TRANSFORMERS

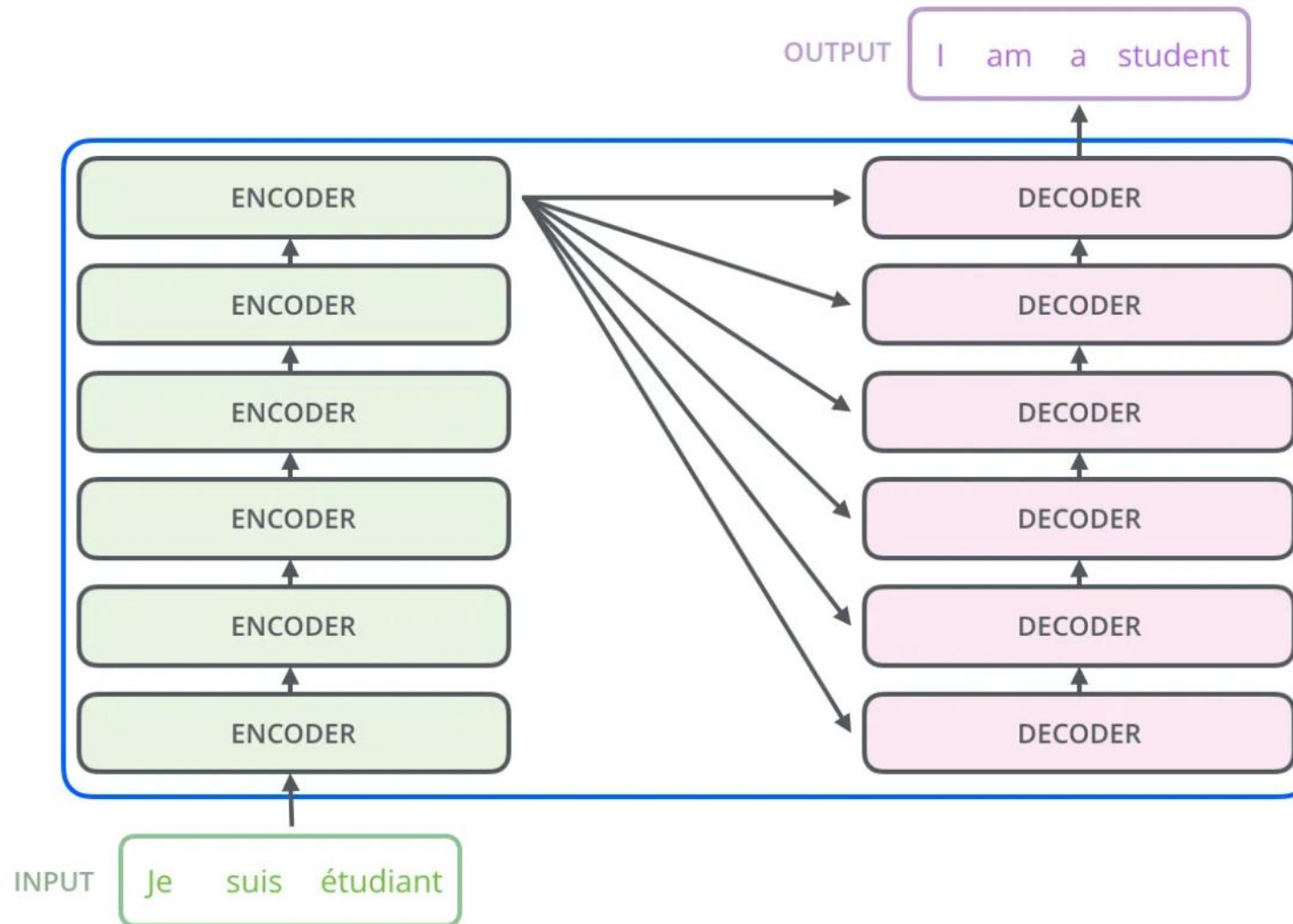
Чтобы решить проблему распараллеливания, Transformers пытаются решить проблему, используя сверточные нейронные сети вместе с моделями внимания. Внимание увеличивает скорость перевода модели из одной последовательности в другую.

Transformer — модель, использующая внимание для увеличения скорости. Более конкретно, он использует само-внимание (**self-attention**).



Внутри Transformer имеет такую же архитектуру, как и предыдущие модели, описанные выше.

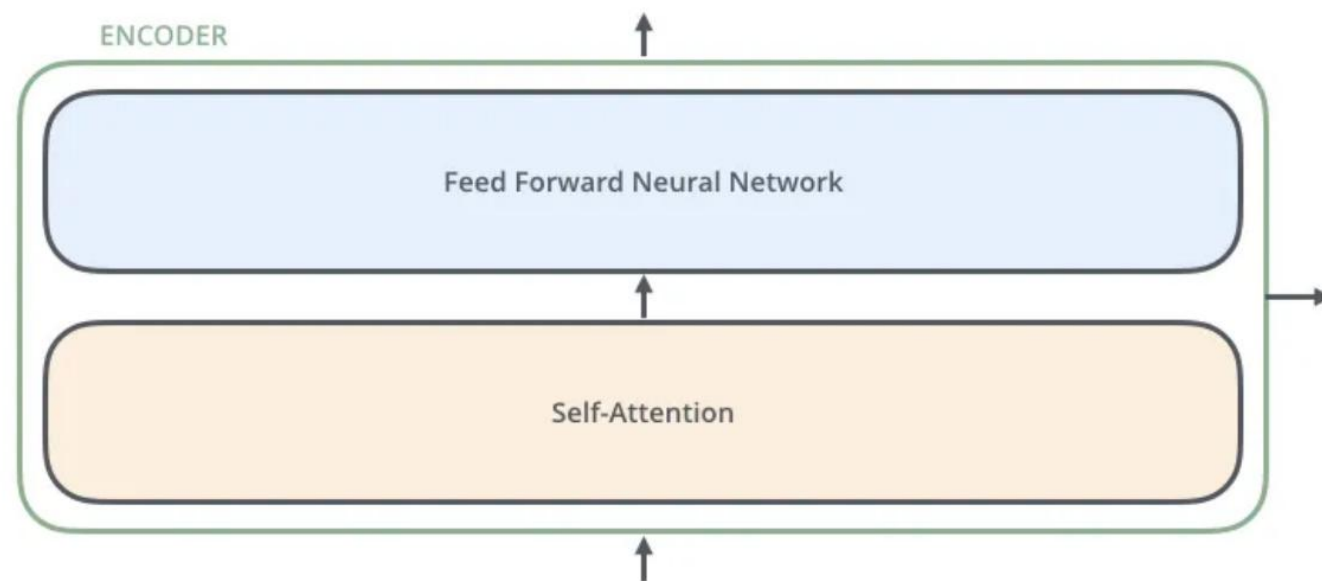
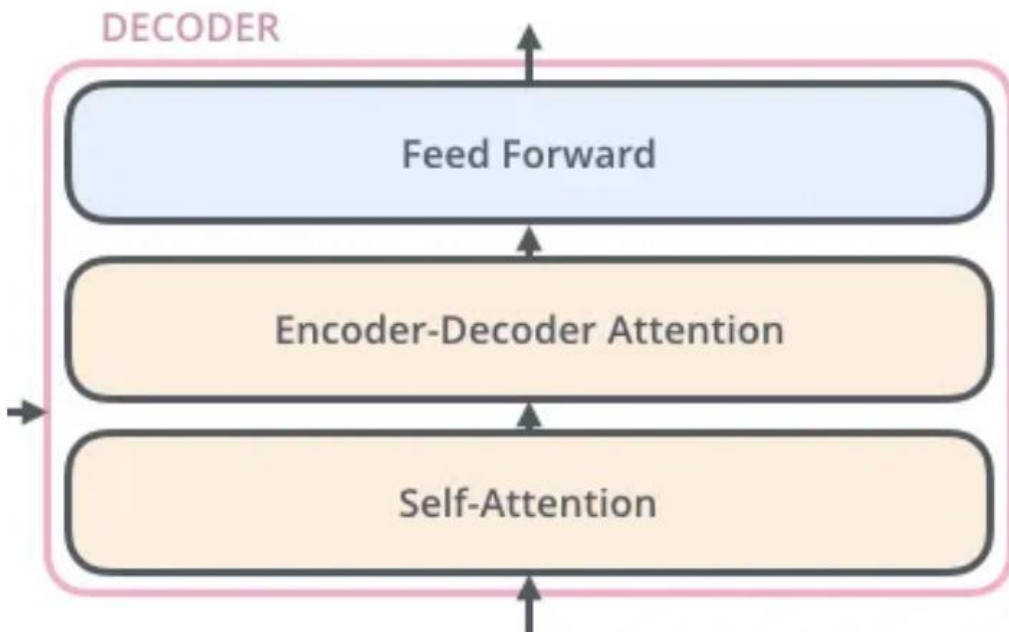
Transformer состоит из шести энкодеров и шести декодеров.



Каждый энкодер очень похож друг на друга. Все кодировщики имеют одинаковую архитектуру.

Декодеры имеют одно и то же свойство, то есть они также очень похожи друг на друга.

Каждый кодер состоит из двух слоев: самовнимания и прямой нейронной сети.



Входные данные кодировщика сначала проходят через слой самоконтроля. Это помогает кодировщику смотреть на другие слова во входном предложении, поскольку он кодирует конкретное слово. У декодера есть оба этих слоя, но между ними находится слой внимания, который помогает декодеру сосредоточиться на соответствующих частях входного предложения.

SELF-ATTENTION

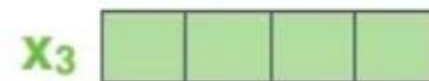
Давайте начнем рассматривать различные векторы/тензоры и то, как они перетекают между этими компонентами, чтобы превратить вход обученной модели в результат. Как и в случае с приложениями НЛП в целом, мы начинаем с преобразования каждого входного слова в вектор с помощью алгоритма встраивания (embedding algorithm).



Je



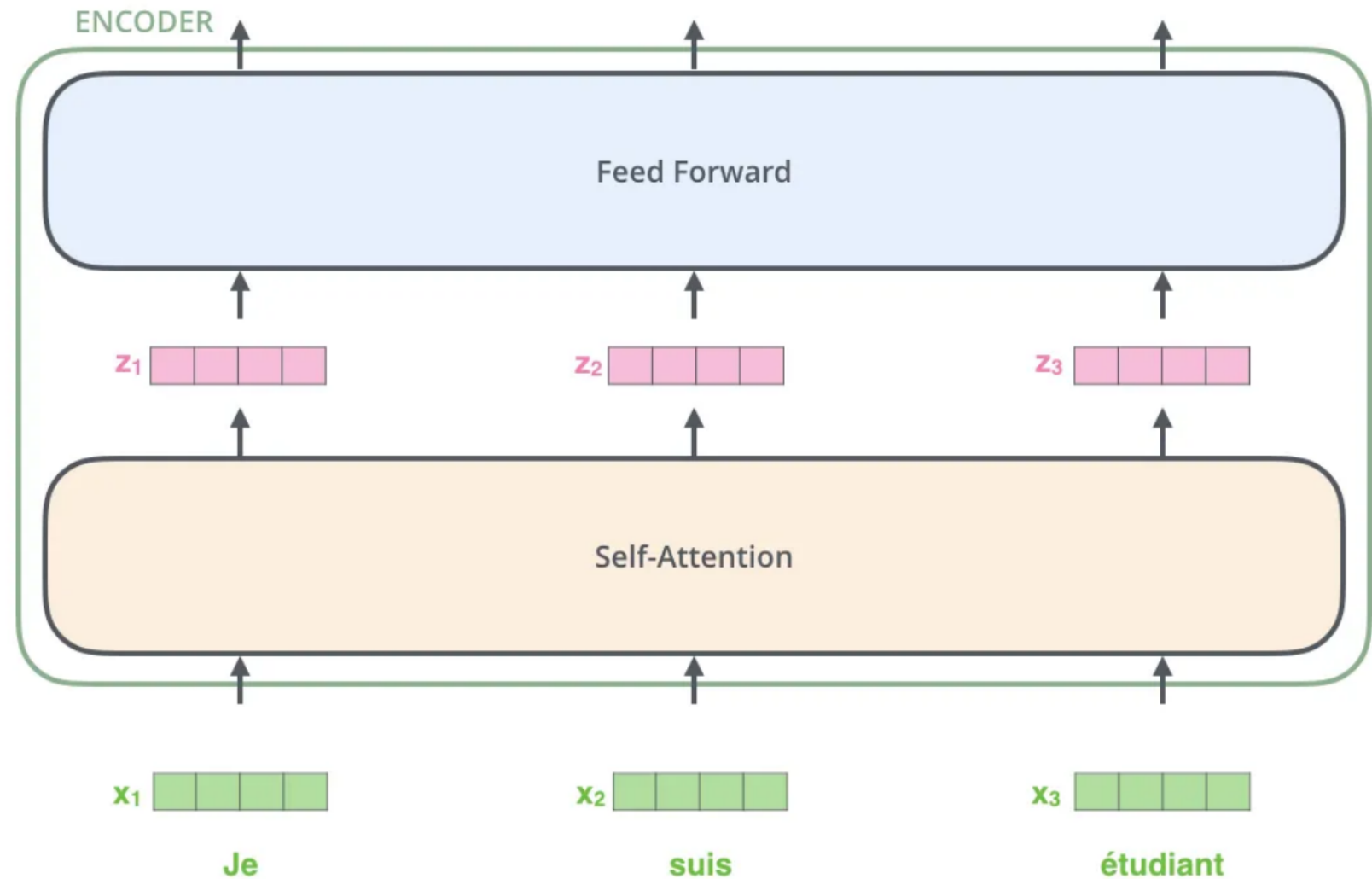
suis

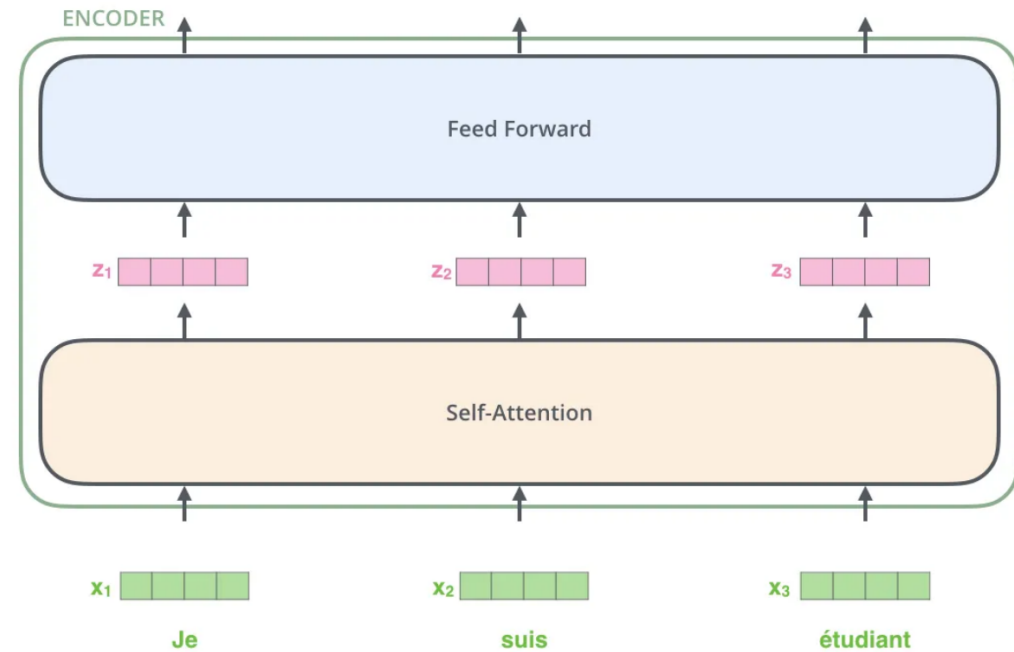


étudiant

Каждое слово встроено в вектор размером 512. Мы будем представлять эти векторы с помощью этих простых прямоугольников. Встраивание происходит только в самом нижнем кодировщике. Абстракция, общая для всех кодировщиков, заключается в том, что они получают список векторов размером 512 каждый.

В нижнем кодировщике это будут вложения слов, но в других кодировщиках это будут выходные данные кодировщика, который находится непосредственно под ним.
После встраивания слов в нашу входную последовательность каждое из них проходит через каждый из двух слоев кодировщика.





Здесь мы начинаем видеть одно ключевое свойство Трансформеров, а именно то, что слово в каждой позиции проходит в кодировщике по своему пути. Между этими путями на уровне внимания к себе существуют зависимости.

Однако уровень прямой связи не имеет таких зависимостей, и поэтому различные пути могут выполняться параллельно при прохождении через уровень прямой связи.

ПОНИМАНИЕ САМО-ВНИМАНИЯ

(внутреннее внимание)

"The animal didn't cross the street because it was too tired"

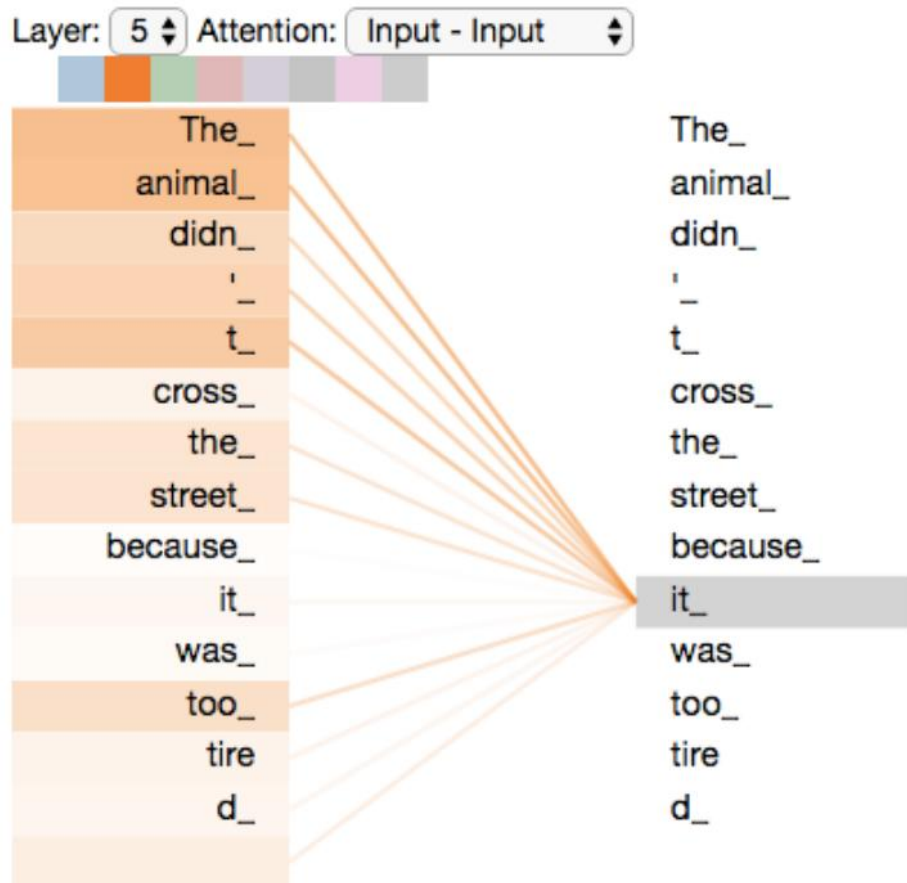
К чему относится «it» в этом предложении? К улице (street) или к животному (animal)? Простой вопрос для человека становится целой проблемой для алгоритма.

Когда модель обрабатывает слово «it», слой внутреннего внимания помогает понять, что «it» относится к «animal».

По мере того как модель обрабатывает каждое слово (каждую позицию во входной последовательности), внутреннее внимание позволяет модели взглянуть на другие позиции входной последовательности и найти подсказку, помогающую лучше закодировать данное слово.

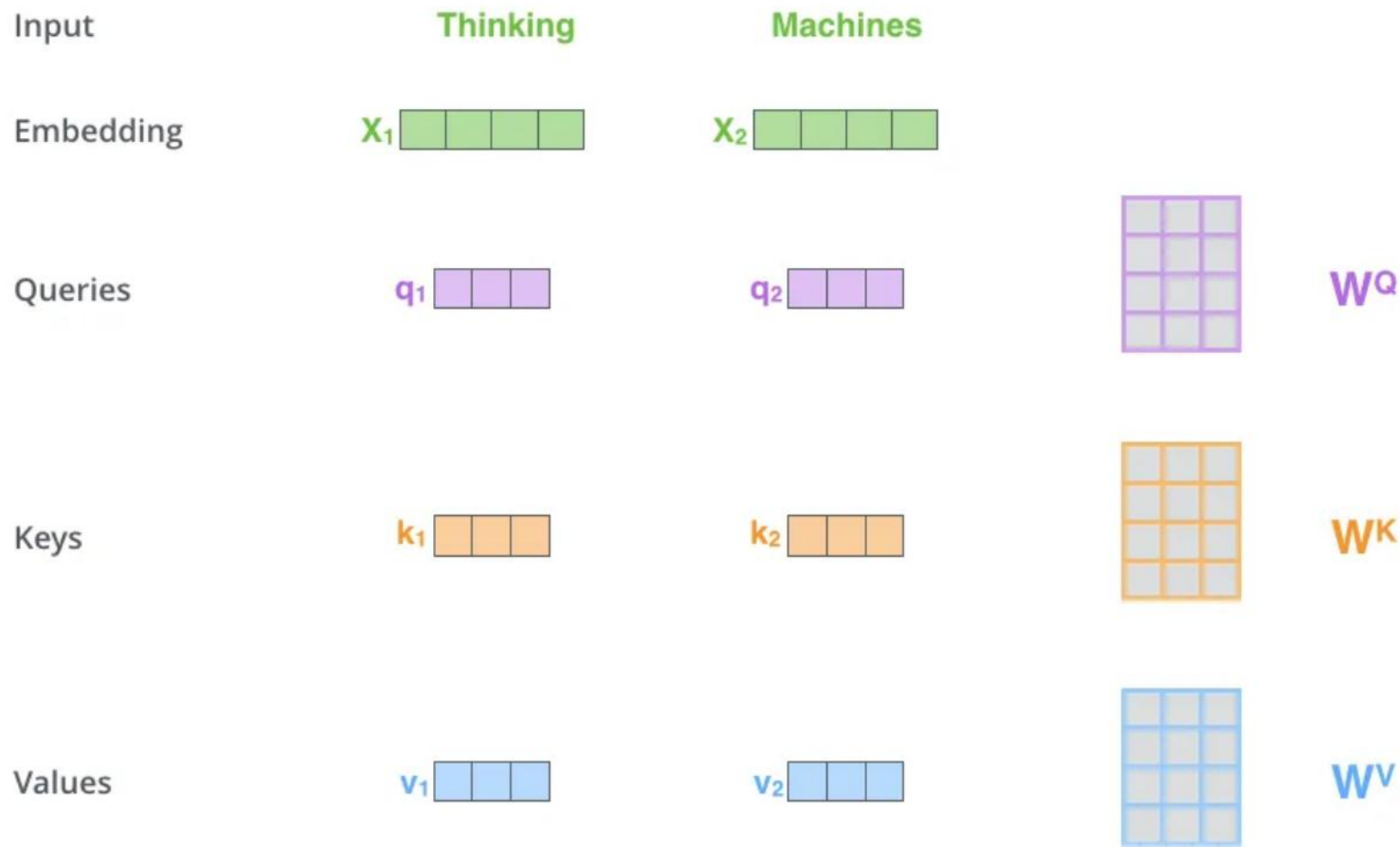
Вспомните, как сохранение скрытого состояния в RNN позволяет включать представление предыдущих слов/векторов, которые уже были обработаны, в текущее обрабатываемое слово.

Механизм внутреннего внимания – это метод, который Трансформер использует для того, чтобы смоделировать «понимание» других релевантных слов при обработке конкретного слова.



Во время кодирования «it» в энкодере #5 (верхний энкодер в стеке), часть механизма внимания фокусируется на «The animal» и использует фрагмент его представления для кодирования «it».

Обратите внимание, что эти новые векторы меньше по размеру, чем вектор вложения. Их размерность равна 64, в то время как векторы ввода/вывода встраивания и кодирования имеют размерность 512. Они НЕ ДОЛЖНЫ быть меньше, это выбор архитектуры, чтобы сделать вычисление внимания (в основном) постоянным.

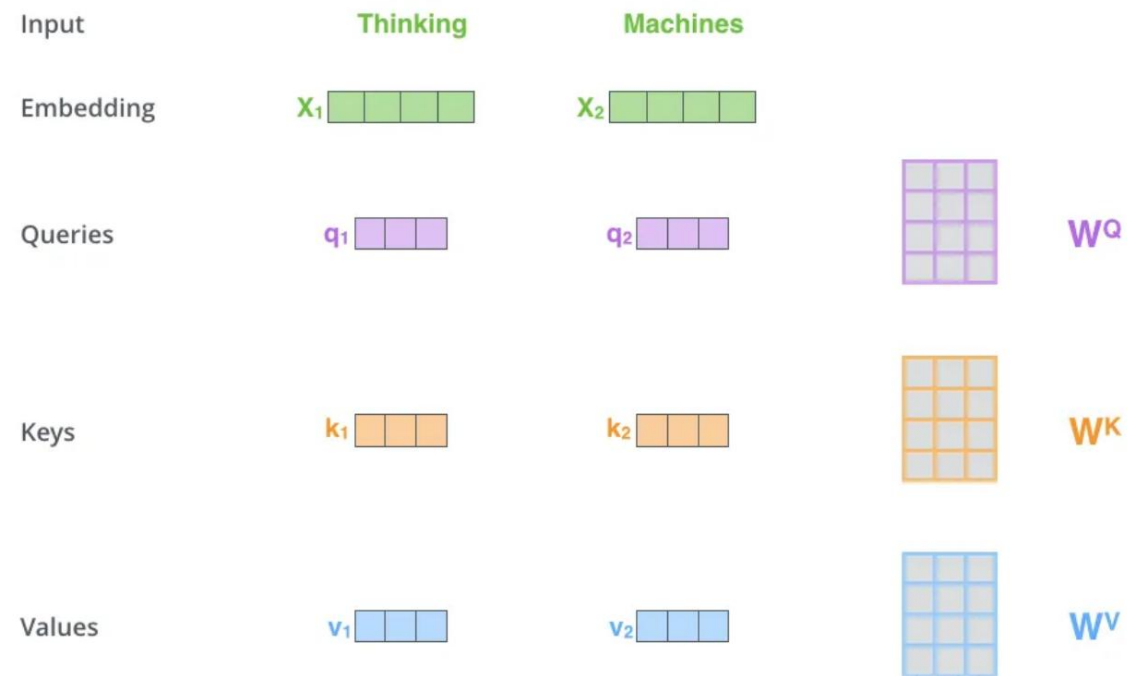


Умножение x_1 на весовую матрицу W^Q дает q_1 , вектор «запроса», связанный с этим словом. В итоге мы создаем «запрос», «ключ» и «значение» проекции каждого слова во входном предложении.

Что такое векторы «запрос», «ключ» и «значение»?

Это абстракции, полезные для расчетов и размышлений о внимании.

Далее в лекции будет понятен их смысл (но это не точно)



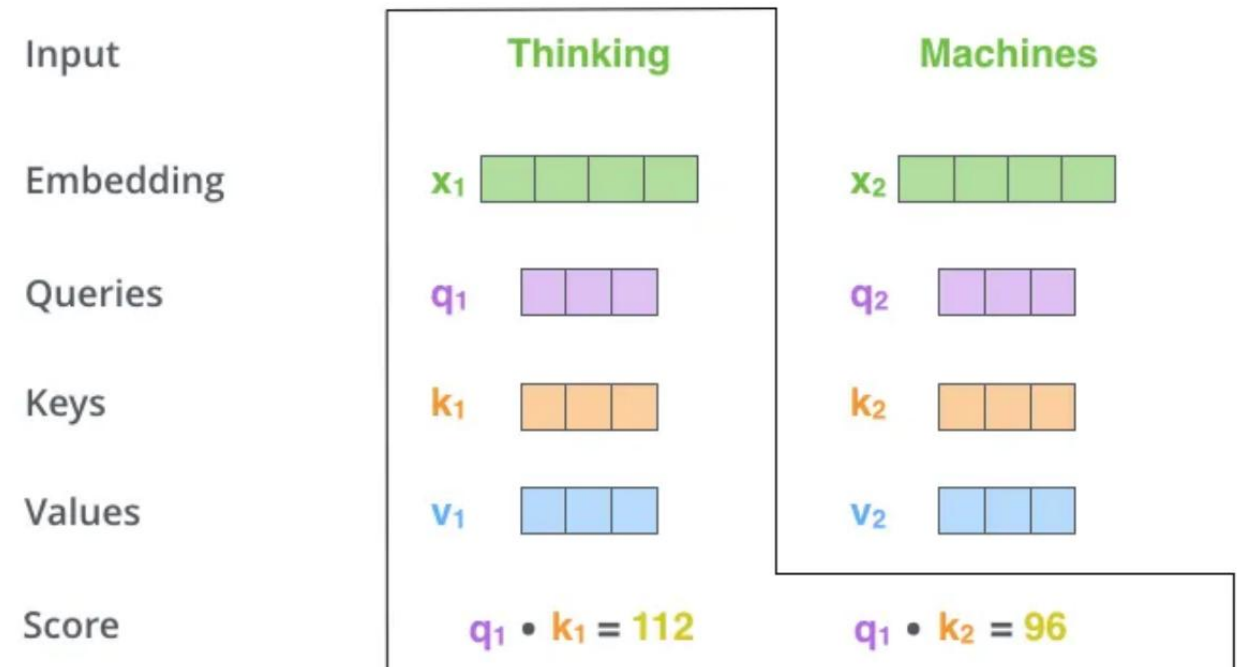
Второй шаг в подсчете само-внимания— подсчет баллов.

Скажем, мы вычисляем само-внимание для первого слова в этом примере «Думаю».

Нам нужно сопоставить каждое слово входного предложения с этим словом. Оценка определяет, сколько внимания нужно уделять другим частям входного предложения, когда мы кодируем слово в определенной позиции.

Оценка рассчитывается путем скалярного произведения вектора запроса на ключевой вектор соответствующего слова, которое мы оцениваем.

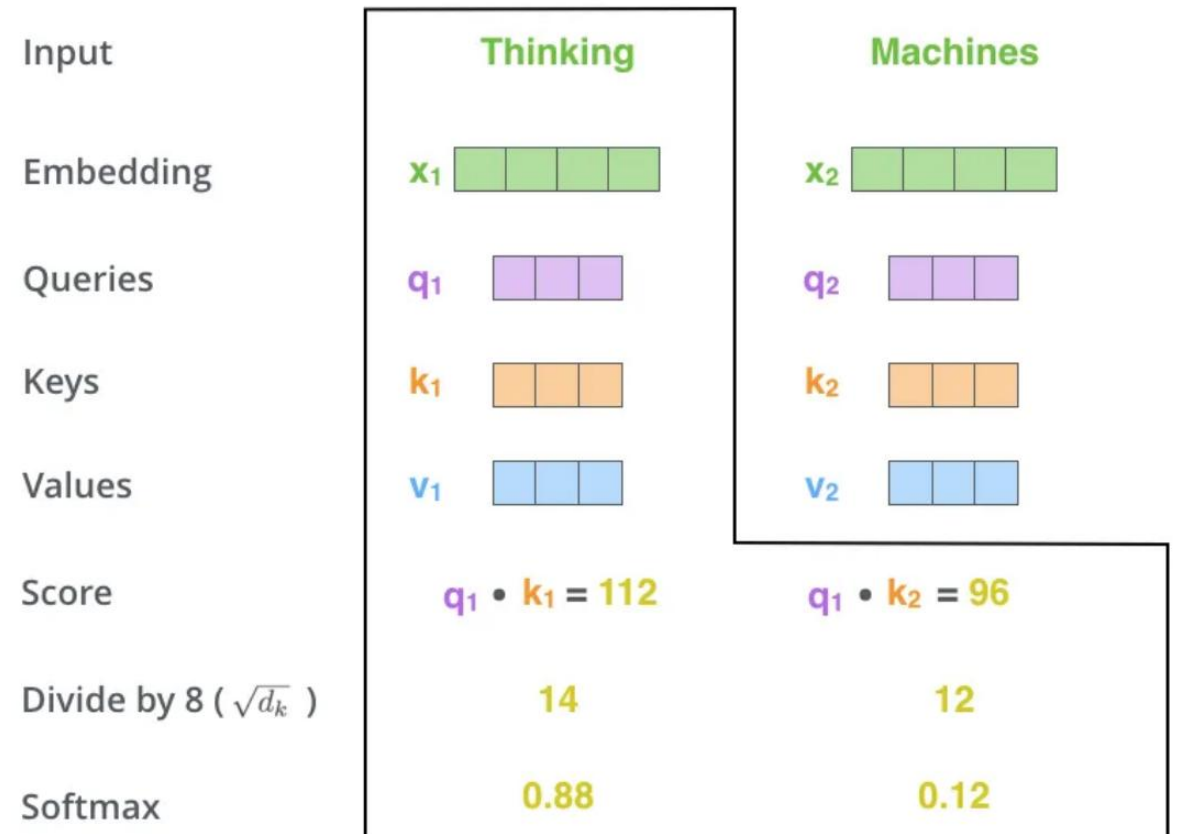
Итак, если мы обрабатываем самовнимание для слова в позиции № 1, первая оценка будет скалярным произведением q_1 и k_1 . Вторая оценка будет скалярным произведением q_1 и k_2 .



Третий и четвертый шаги — разделить баллы на 8 (квадратный корень из размерности ключевых векторов — 64. Это приводит к более стабильным градиентам. Здесь могут быть и другие возможные значения, но это default), затем передайте результат через операцию softmax.

Softmax нормализует оценки, чтобы все они были положительными и в сумме давали 1.

Эта оценка softmax определяет, насколько каждое слово будет выражено в этой позиции. Очевидно, что слово в этой позиции будет иметь наивысший балл softmax, но иногда полезно обратить внимание на другое слово, имеющее отношение к текущему слову.

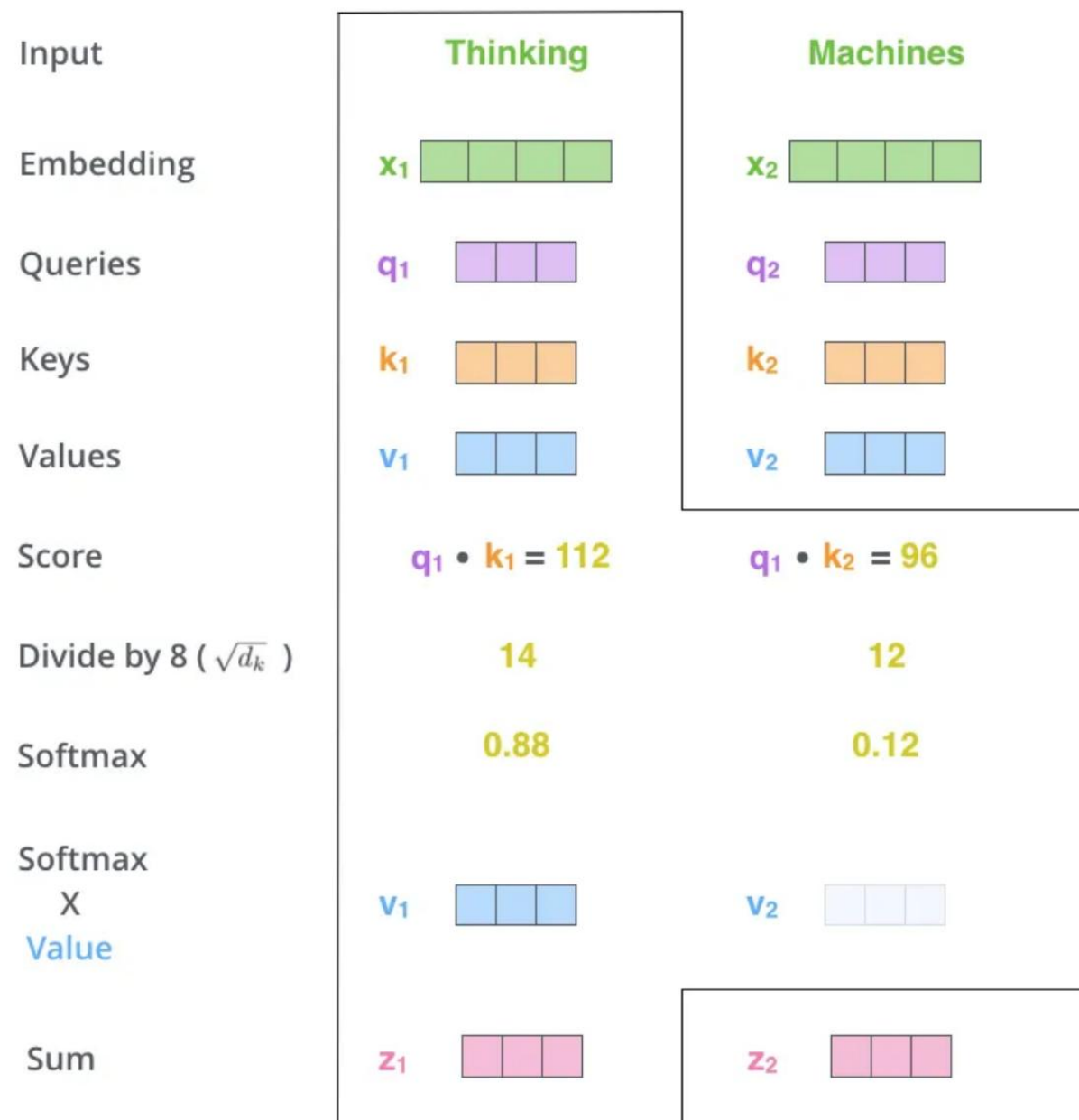


Пятый шаг — умножить каждый вектор значений на оценку softmax (при подготовке к их суммированию). Интуиция здесь состоит в том, чтобы сохранить нетронутыми значения слов, на которых мы хотим сосредоточиться, и заглушить нерелевантные слова (например, умножив их на крошечные числа, такие как 0,001).

Шестой шаг заключается в суммировании взвешенных векторов значений. Это производит вывод слоя внутреннего внимания в этой позиции (для первого слова).

На этом расчет собственного внимания заканчивается. Результирующий вектор — это тот, который мы можем отправить в нейронную сеть с прямой связью.

Однако в реальной реализации этот расчет выполняется в матричной форме для более быстрой обработки.



МАТРИЧНЫЕ ВЫЧИСЛЕНИЯ ВНУТРЕННЕГО ВНИМАНИЯ

Первый этап – вычислить матрицы запроса, ключа и значения. Это делается с помощью формирования из эмбедингов матрицы X и ее умножения на матрицы весов, которые мы обучили (WQ , WK , WV).

$$\begin{matrix} X \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^Q \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} Q \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} X \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^K \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} K \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} X \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^V \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} V \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

Каждая строка в матрице X соответствует слову во входящем предложении. Мы снова видим разницу в размерах векторов эмбедингов (512, или 4 квадратика на рисунке) и векторов $q/k/v$ (64, или 3 квадратика).

Наконец, поскольку мы имеем дело с матрицами, мы можем сжать этапы 2-6 в одну формулу для вычисления выхода слоя внутреннего внимания.

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$
$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

MULTIHEAD ATTENTION

Трансформеры в основном работают так. Есть еще несколько деталей, которые улучшают их работу. Например, вместо того, чтобы обращать внимание друг на друга только в одном измерении, Трансформеры используют концепцию множественного внимания (Multihead attention).

Идея заключается в том, что всякий раз, когда вы переводите слово, вы можете уделять разное внимание каждому слову в зависимости от типа вопроса, который вы задаете.

1.Повышается способность модели фокусироваться на разных позициях. Да, в примере выше, z_1 содержит немного от всех других кодировок, но он не может доминировать над самим словом.

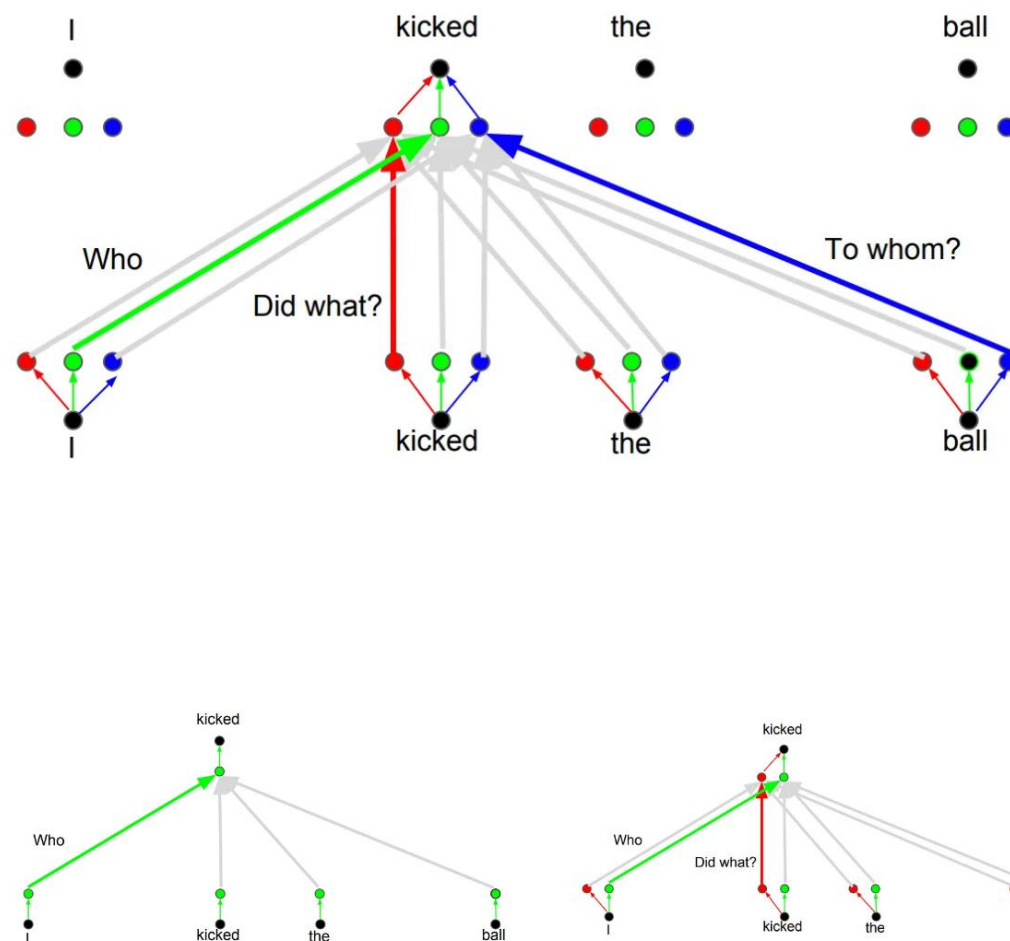
2.В случае с переводом предложения вроде «The animal didn't cross the street because it was too tired», мы хотим знать, к какому слову относится «it».

3.Слой внимания снабжается множеством «подпространств представлений» (representation subspaces). Как мы увидим далее, с помощью множественного внимания у нас есть не один, а множество наборов матриц запроса/ключа/значения (Трансформер использует 8 «голов» внимания, так что в итоге у нас получается 8 наборов для каждого энкодера/декодера).

Каждый из этих наборов создается случайным образом.

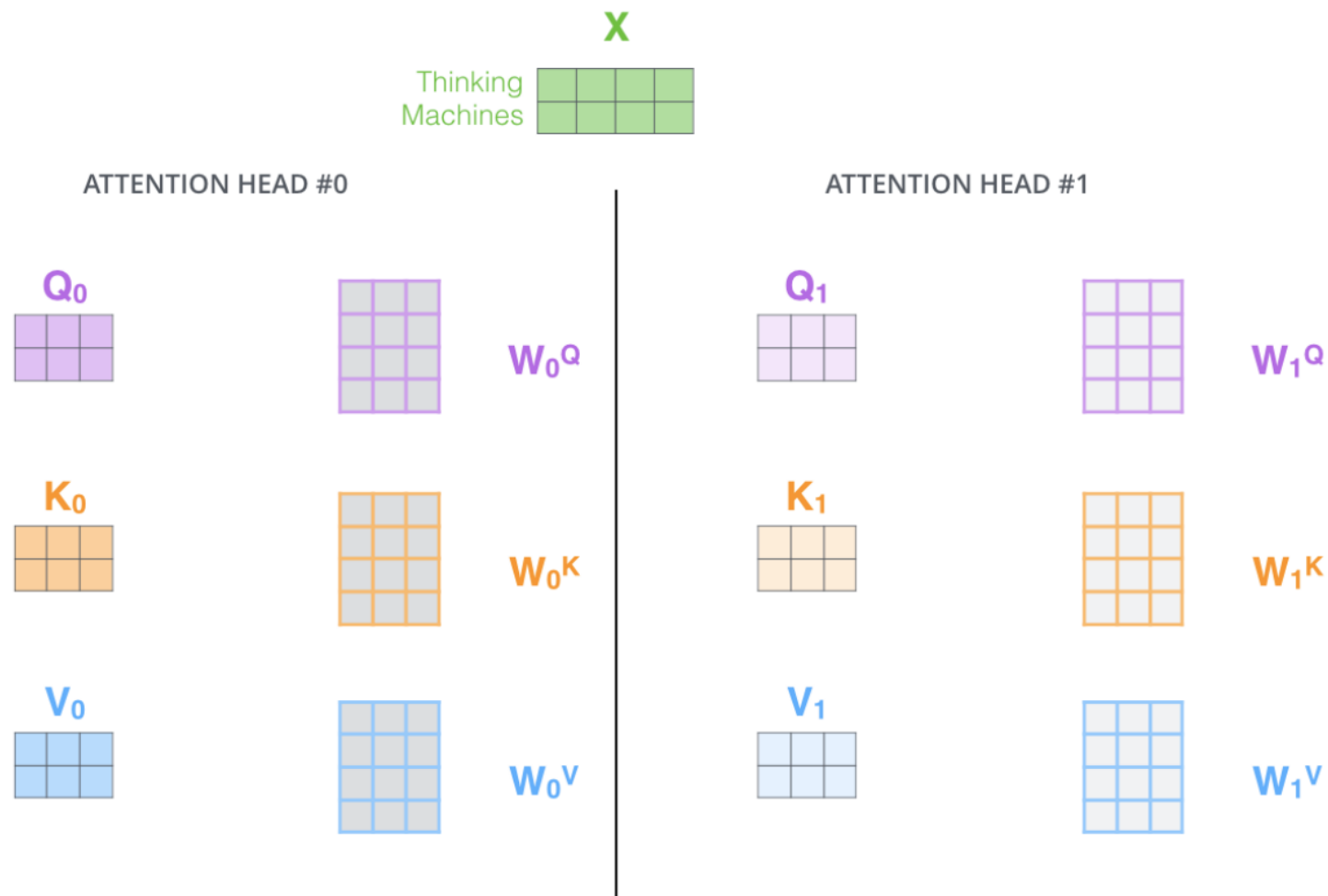
Далее после обучения каждый набор используется для отображения входящих эмбеддингов (или векторов с нижестоящих энкодеров/декодеров) в разных подпространствах представлений.

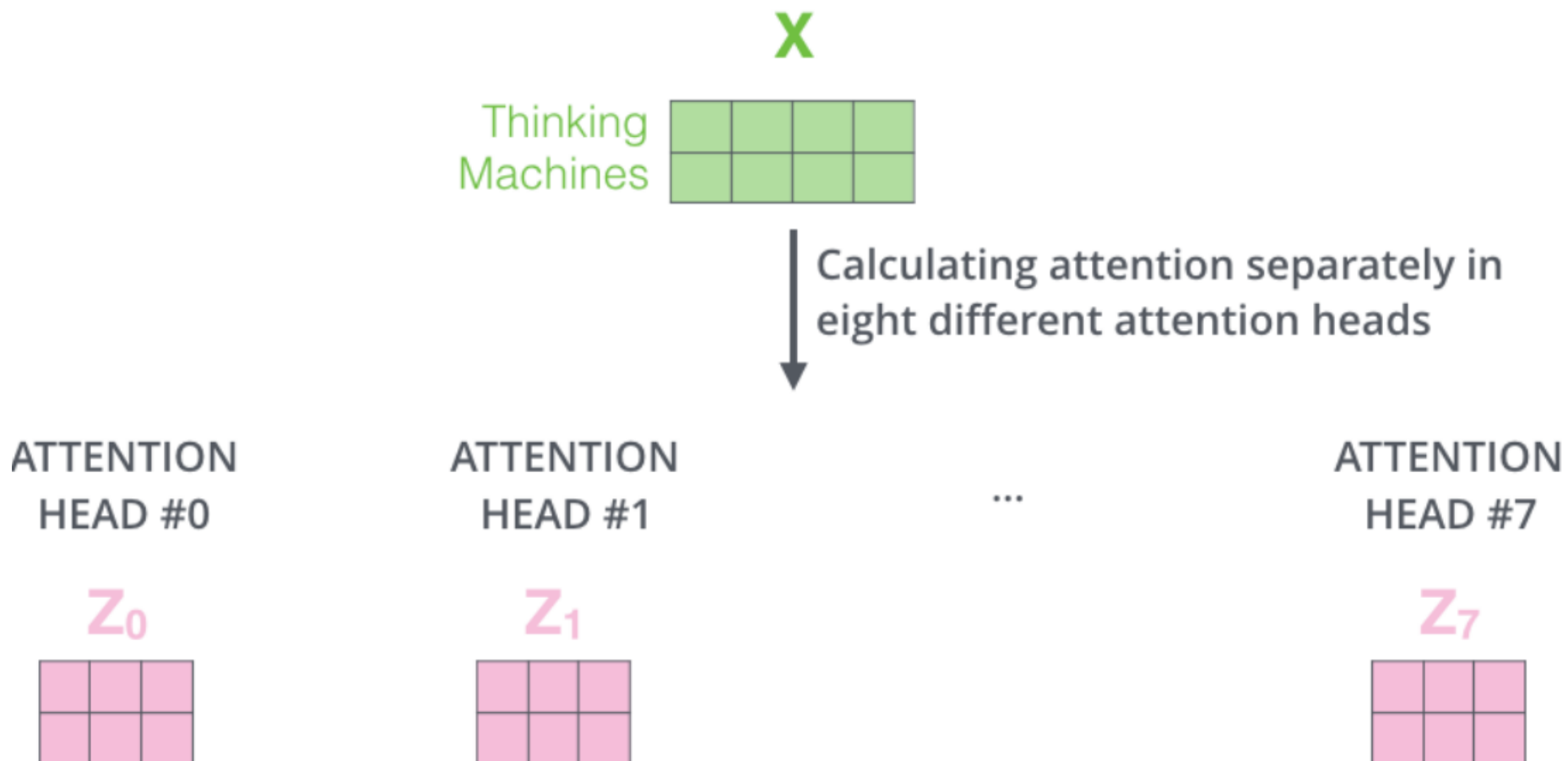
Например, всякий раз, когда вы переводите «ударил ногой» в предложении «I kicked the ball», вы можете спросить «Who kicked». В зависимости от ответа перевод слова на другой язык может измениться. Или задайте другие вопросы, например «Did what?» и т. д.



В случае множественного внимания, мы располагаем отдельными $WQ/WK/WV$ матрицами весов для каждой «головы», что в результате дает разные $Q/K/V$ матрицы. Как мы делали ранее, умножаем X на $WQ/WK/WV$ матрицы для получения $Q/K/V$ матриц.

Сделав те же вычисления внутреннего внимания, что мы описали выше, 8 раз с разными матрицами весов, в результате получим 8 разных Z матриц.





Проблема. Слой сети прямого распространения не ожидает, что к нему поступит 8 матриц – он ждет всего одну (вектор для каждого слова), в которую нам и необходимо сжать полученные Z матрицы.

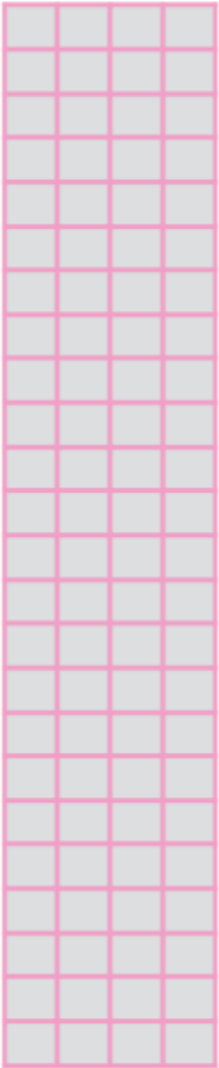
Как это сделать? Конкатенировать и затем умножить их на дополнительные веса матрицы W^O

1) Concatenate all the attention heads



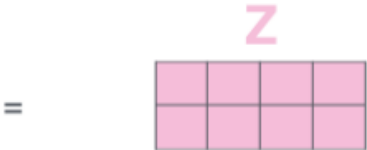
2) Multiply with a weight matrix W^O that was trained jointly with the model

x



W^O

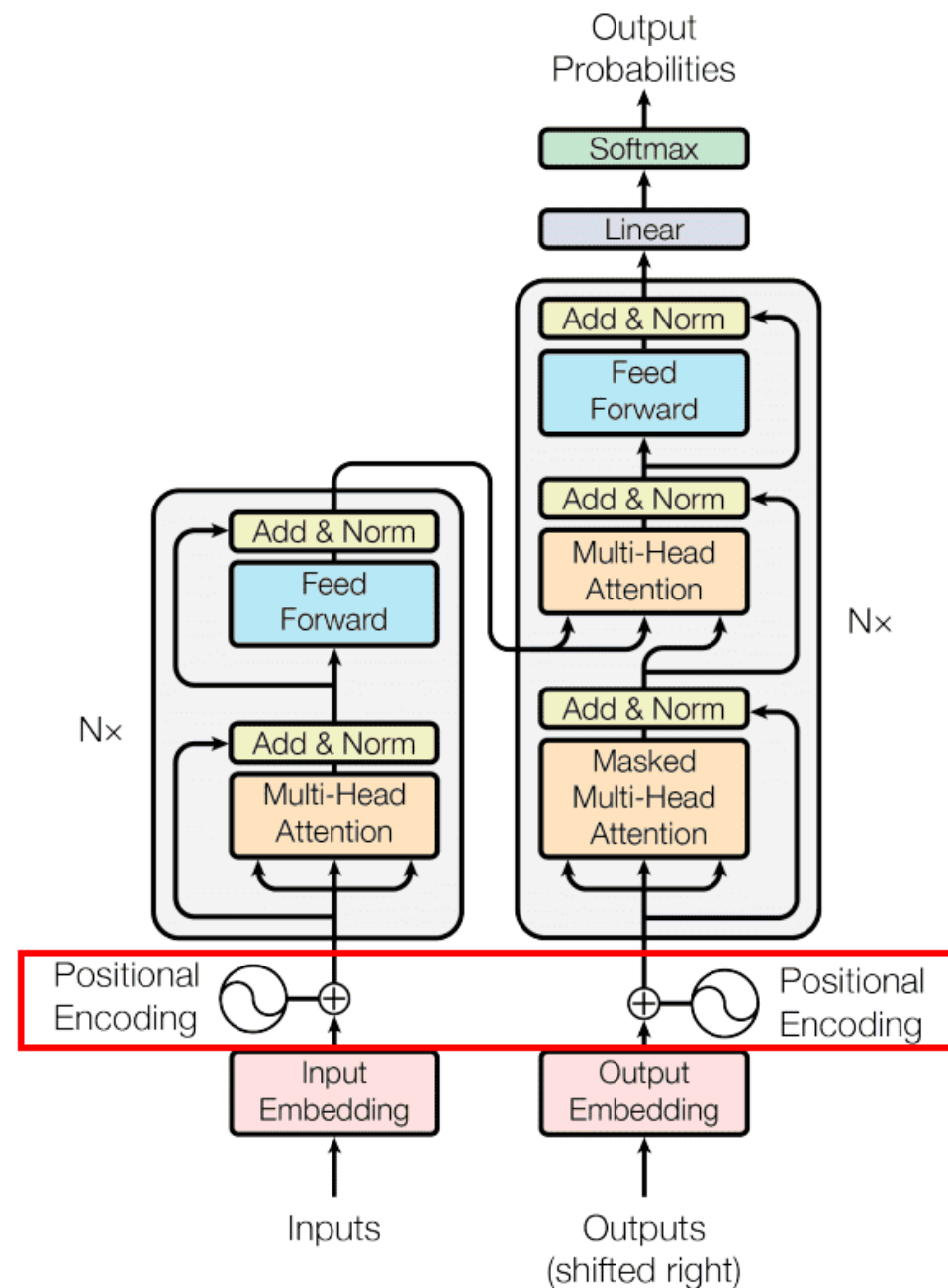
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



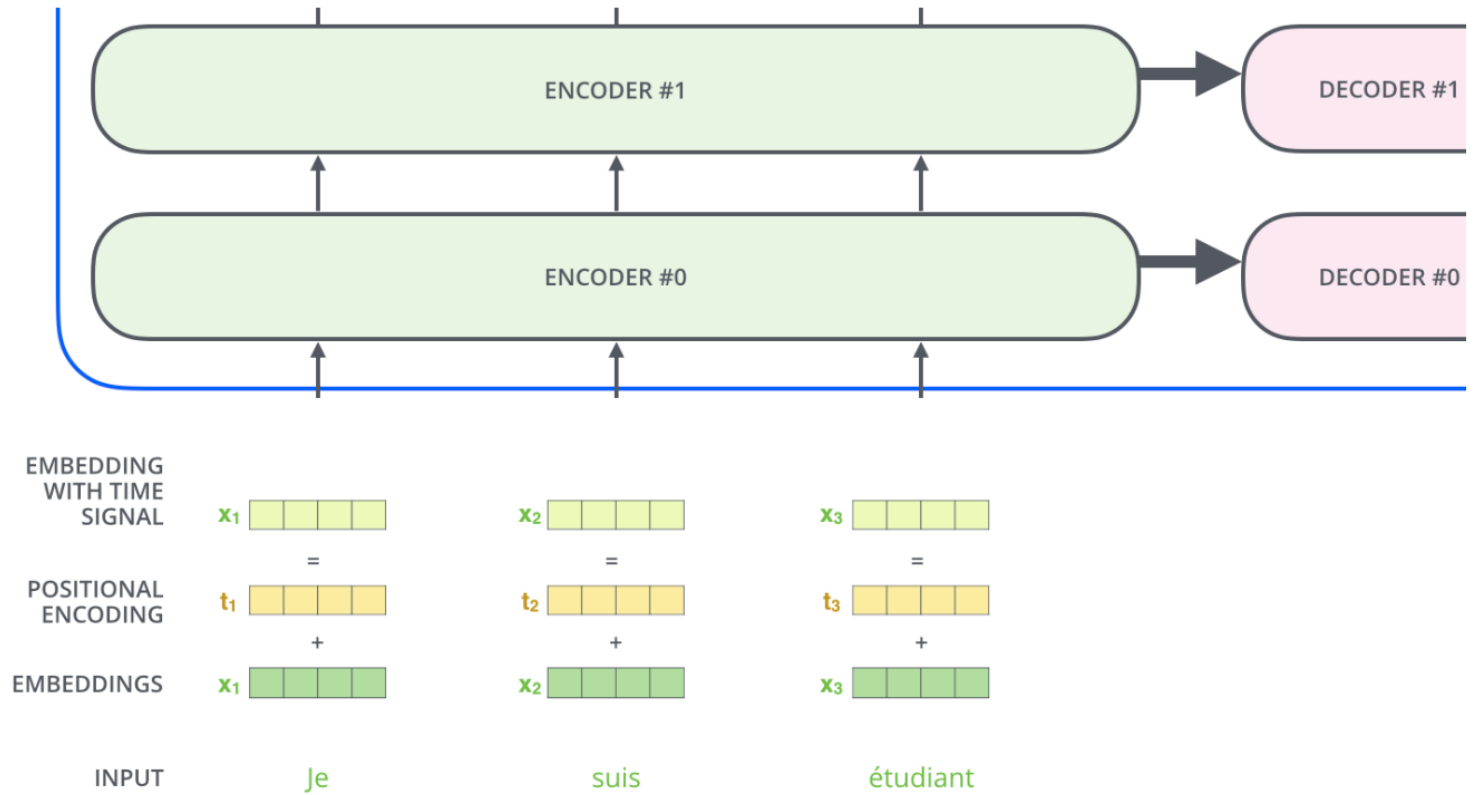
POSITIONAL ENCODING

Позиционное кодирование

Еще одним важным шагом в Transformer является добавление позиционного кодирования при кодировании каждого слова. Кодирование положения каждого слова имеет значение, поскольку положение каждого слова имеет отношение к переводу.



Для решения этой проблемы в Трансформеры добавляют вектор в каждый входящий эмбеддинг. Эти векторы имеют определенный шаблон, который модель запоминает и который помогает определить позицию каждого слова или расстояние между разными словами в предложении. Интуиция здесь состоит в том, что добавление этих значений к эмбеддингам создает осмысленные расстояния между векторами эмбеддингов в процессе их проецирования в Q/K/V векторы и скалярного произведения при вычислении внимания.



РАСЧЕТ СТАТИЧЕСКОГО ПОЗИЦИОНИРОВАНИЯ

Можно использовать функции $\sin$ и $\cos$ для вычисления значения для каждого входного вектора.

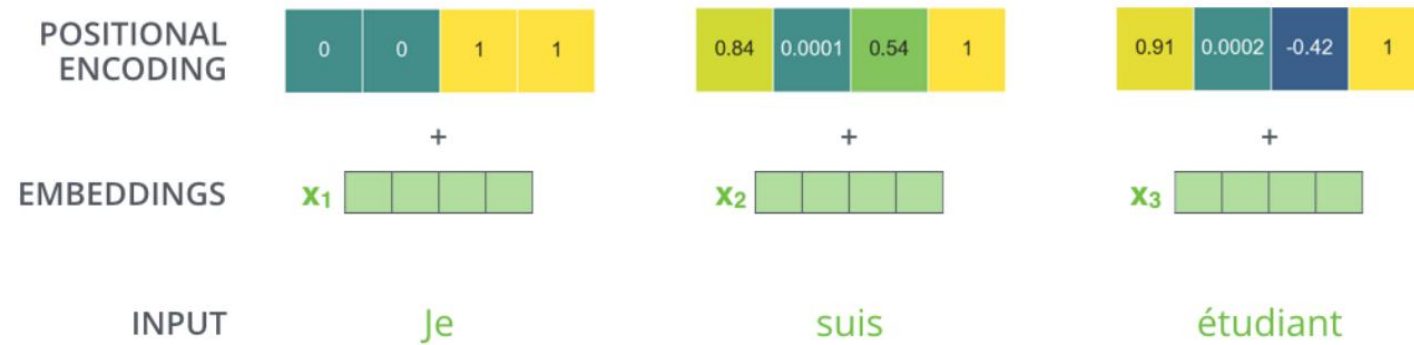
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos - position of the vector
- i - index within the vector
- d_{model} - dimension of the input

Для того, чтобы модель понимала порядок слов, мы добавляем векторы позиционного кодирования, значения которых следуют определенному шаблону.

Если мы считаем, что эмбединг имеет размерность 4, то реальные позиционные кодировки будут выглядеть так:



Transformer vs RNN: ключевые отличия

Параллелизм

RNN обрабатывает токены строго последовательно — шаг за шагом. Transformer обрабатывает всю последовательность за один проход, что даёт кратное ускорение на GPU/TPU.

Дальние зависимости

RNN «забывает» далёкий контекст из-за затухания градиента. Transformer напрямую вычисляет attention между любыми двумя позициями — расстояние не имеет значения.

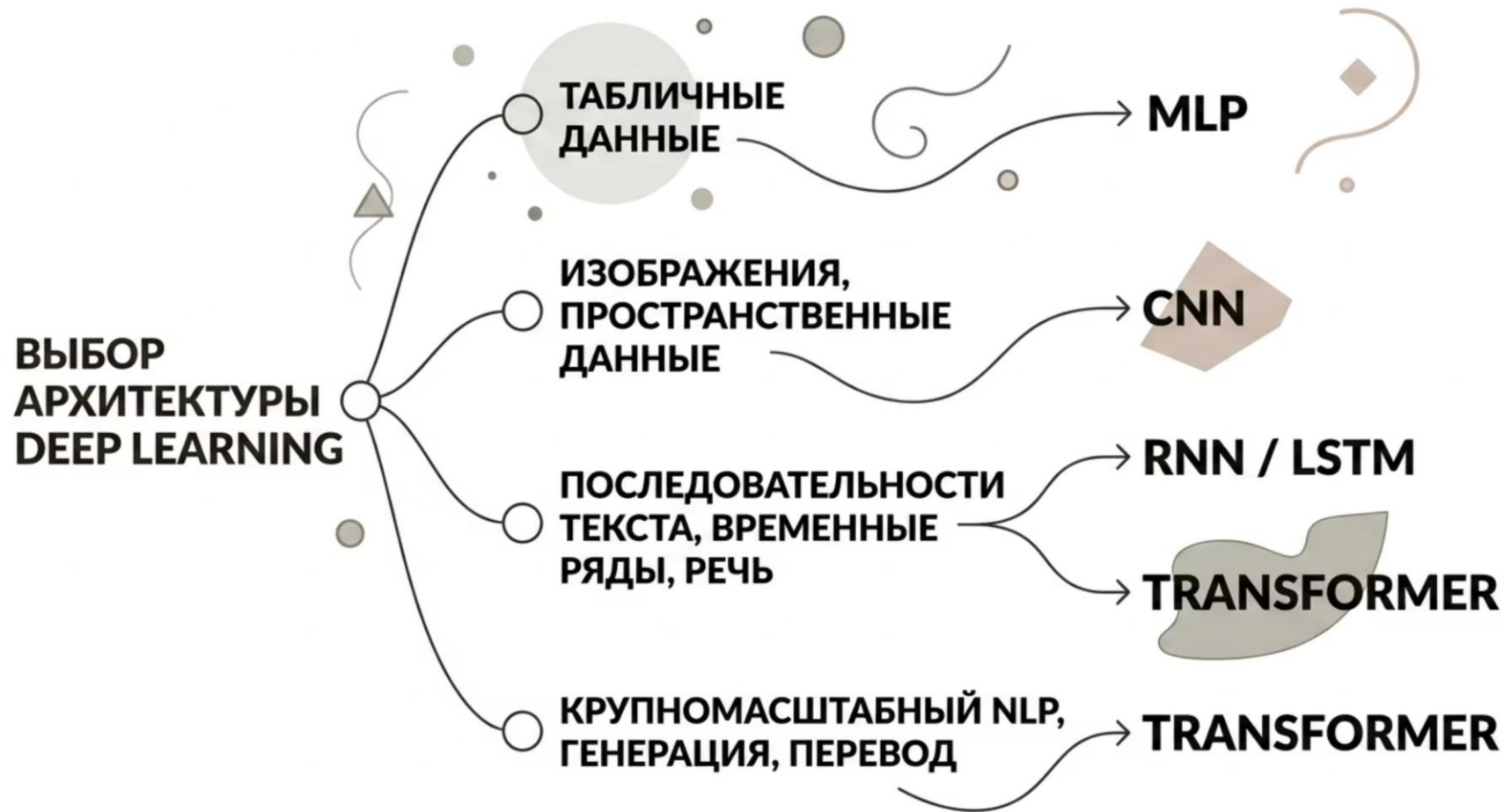
Масштабируемость

Transformer масштабируется до миллиардов параметров (GPT-4, Gemini). RNN/LSTM практически вытеснены Transformer в задачах NLP, хотя остаются актуальны для потоковых данных.

Сводная карта архитектур глубокого обучения

Архитектура	Ключевой механизм	Типичные задачи	Главный нюанс
MLP	Полносвязные слои + активации	Табличные данные, классификация/регрессия	Не учитывает пространство и время
CNN	Свёртки + пулинг	Изображения, детекция, сегментация	Трансляционная инвариантность
RNN/LSTM	Рекуррентное состояние + гейты	Текст, временные ряды, речь	LSTM решает затухание градиента
Transformer	Self-attention	NLP, генерация, Vision Transformer	Параллелизм, прямые дальние связи

Как выбрать архитектуру под задачу





РАЗДЕЛ 4

С чего начать на практике: 3 шага

Практический план для начинающего ML-инженера, позволяющий быстро получить первый работающий результат и выстроить правильные привычки с самого начала.

Шаг 1: Выбрать библиотеку

PyTorch

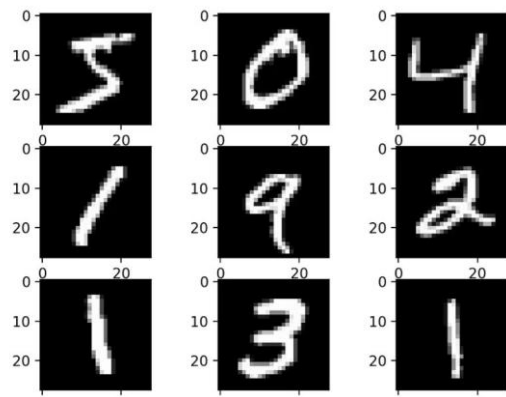
Рекомендуется для глубокого обучения. Динамический граф вычислений делает отладку интуитивной — можно использовать обычный Python-дебаггер. Богатая документация, огромное сообщество, де-факто стандарт в академических исследованиях. Простой синтаксис для первых экспериментов с нейронными сетями.

scikit-learn

Рекомендуется для классических моделей ML. Единый API для всех алгоритмов (fit/predict/transform), встроенные инструменты для предобработки данных, кросс-валидации и оценки метрик. Идеальна для начала: позволяет сфокусироваться на понимании алгоритмов, а не на деталях реализации.

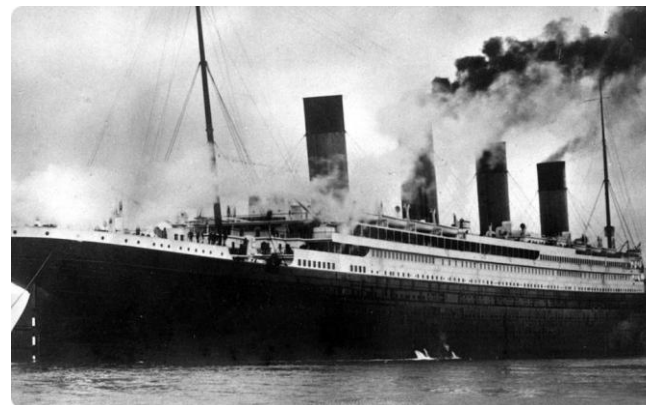
- ❏ Обе библиотеки имеют богатую документацию, большое сообщество и простой синтаксис для первых экспериментов. Начните с scikit-learn для классических моделей, добавьте PyTorch, когда перейдёте к нейронным сетям.

Шаг 2: Выбрать простой датасет



MNIST — рукописные цифры

70 000 изображений цифр 0–9 (28×28 пикселей). Идеален для задач классификации изображений. Небольшой, чистый, хорошо изученный — позволяет сфокусироваться на понимании пайплайна, а не на борьбе с грязными данными.



Titanic / Iris — табличные данные

Классические датасеты для задач с табличными данными. Titanic — бинарная классификация (выжил / не выжил). Iris — многоклассовая классификация цветков. Оба небольшие, чистые и хорошо документированные.

Шаг 3: Обучить базовую модель (baseline)

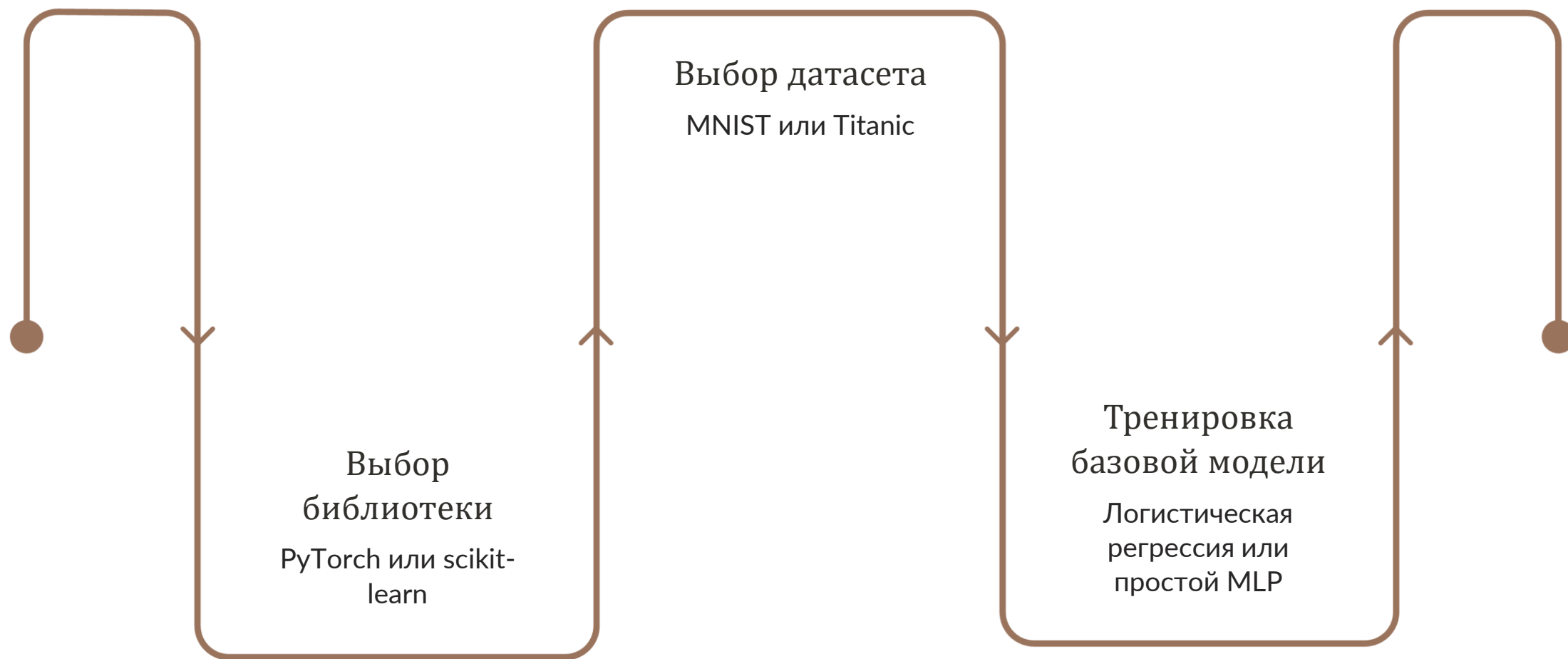
Почему начинать с простого?

Начните с логистической регрессии или простого MLP из 1–2 скрытых слоёв **прежде, чем переходить к сложным архитектурам**. Это даёт понятную точку отсчёта, относительно которой можно измерять улучшение более сложных моделей. Без baseline невозможно понять, действительно ли сложная модель лучше или просто сложнее.

Что даёт baseline

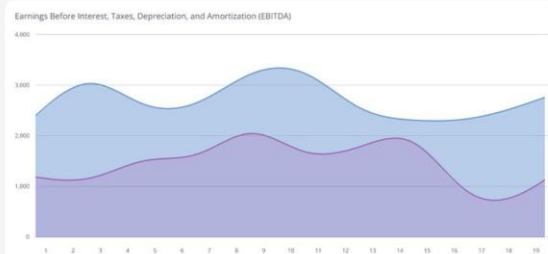
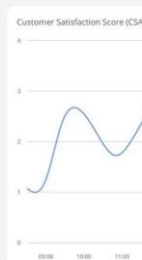
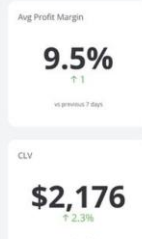
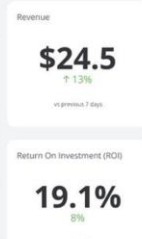
- Проверка корректности пайплайна данных
- Понимание сложности задачи
- Точка отсчёта для сравнения улучшений
- Быстрый первый результат, который можно показать
- Выявление проблем с данными до усложнения модели

Три шага к первой ML-модели



Performance Management Dashboard

Auto date range: ☒ This Week Services: All Posts: All



РАЗДЕЛ 5

Метрики оценки качества модели

Правильный выбор метрики — не менее важен, чем выбор архитектуры. Метрика должна отражать реальную бизнес-цель, а не быть выбрана «по умолчанию».

Матрица ошибок — основа метрик классификации

Четыре элемента confusion matrix

- **TP (True Positive)** — модель предсказала «положительный», и это верно
- **TN (True Negative)** — модель предсказала «отрицательный», и это верно
- **FP (False Positive)** — модель предсказала «положительный», но это ошибка (ложная тревога)
- **FN (False Negative)** — модель предсказала «отрицательный», но это ошибка (пропуск)

Почему это важно

Все ключевые метрики классификации строятся на основе этих четырёх чисел. Понимание матрицы ошибок позволяет выбрать метрику, соответствующую реальной стоимости ошибок в конкретной задаче: иногда ложная тревога дороже пропуска, иногда — наоборот.

Например, в медицинской диагностике пропуск больного (FN) значительно опаснее ложной тревоги (FP), что диктует выбор метрики в пользу Recall.

Accuracy — доля правильных предсказаний

Формула

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Доля правильных предсказаний от общего числа примеров.

Когда использовать и когда нет

Accuracy интуитивно понятна и хорошо работает при **сбалансированных классах**. Однако она плохо подходит для несбалансированных данных: при 95% одного класса модель, всегда предсказывающая этот класс, получит accuracy = 95%, оставаясь при этом полностью бесполезной. Высокая accuracy может маскировать полностью бесполезную модель.

Precision и Recall — точность и полнота

Precision (Точность)

$$\text{Precision} = \frac{TP}{TP + FP}$$

Доля верных срабатываний среди всех предсказанных положительных. **Важна, когда цена ложного срабатывания высока** — например, в спам-фильтре: лучше пропустить спам, чем заблокировать важное письмо.

Recall (Полнота)

$$\text{Recall} = \frac{TP}{TP + FN}$$

Доля найденных положительных случаев среди всех реально положительных. **Важна, когда цена пропуска положительного случая высока** — например, в диагностике заболеваний: лучше лишний раз проверить, чем пропустить больного.

F1-score — баланс между Precision и Recall

Формула

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Гармоническое среднее precision и recall. Даёт единую сбалансированную метрику, которая высока только тогда, когда обе составляющие высоки.

Когда использовать F1

F1-score — метрика выбора при **несбалансированных классах**, когда нет явного приоритета между precision и recall. Гармоническое среднее «штрафует» за большой разрыв между двумя метриками сильнее, чем арифметическое: F1 = 0.5 при Precision=1.0, Recall=0.33 — это честное отражение того, что модель находит лишь треть положительных примеров.

Метрики регрессии: MSE, RMSE, R²

MSE — Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$

Среднее квадратов ошибок. Сильно штрафует большие ошибки из-за возведения в квадрат. Чувствителен к выбросам.

RMSE — Root Mean Squared Error

Квадратный корень из MSE. Возвращает метрику в единицы измерения целевой переменной, что упрощает интерпретацию: RMSE = 50 000 рублей понятнее, чем MSE = 2 500 000 000.

R² — коэффициент детерминации

Доля дисперсии целевой переменной, объяснённая моделью. Значения от 0 до 1: чем ближе к 1, тем лучше модель объясняет данные. R² = 0 означает, что модель не лучше предсказания среднего значения.

Сводная таблица метрик классификации

Метрика	Формула	Когда важна	Ограничение
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Сбалансированные классы	Маскирует проблемы при дисбалансе
Precision	$TP/(TP+FP)$	Дорогие ложные срабатывания	Игнорирует FN
Recall	$TP/(TP+FN)$	Дорогие пропуски (медицина)	Игнорирует FP
F1-score	$2 \cdot P \cdot R / (P + R)$	Несбалансированные классы	Не учитывает TN

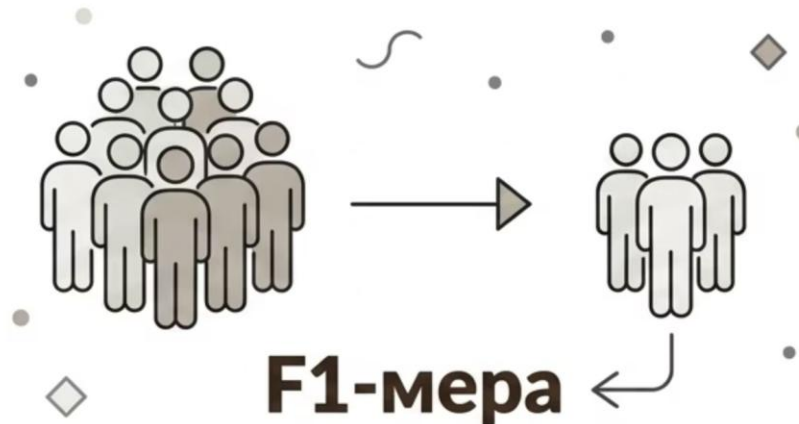


Практическая рекомендация по выбору метрики

❏ Всегда выбирайте метрику в соответствии с бизнес-задачей, а не используйте ассурасу по умолчанию.

Для несбалансированных данных (например, 95% одного класса) высокая ассурасу может маскировать полностью бесполезную модель. Модель, всегда предсказывающая большинство класс, получит $\text{accuracy} = 95\%$ — при этом она не находит ни одного примера меньшинства. F1-score или recall дадут более честную картину качества в таких случаях.

Выбор метрики по типу задачи



Итоговая карта предметной области ML

Парадигмы

Supervised (размеченные данные), Unsupervised (поиск структуры), Reinforcement (награда за действия). Выбор парадигмы определяет всё остальное.

Глубокое обучение

MLP (таблицы), CNN (изображения), RNN/LSTM (последовательности), Transformer (NLP и не только). Выбирайте под структуру данных.

Классические модели

Линейная регрессия, логистическая регрессия, деревья решений, k-NN. Всегда начинайте с них как с baseline.

Метрики

Accuracy, Precision, Recall, F1 — для классификации. MSE, RMSE, R^2 — для регрессии. Выбирайте по бизнес-задаче, не по умолчанию.